

Technical Specification Document

SISTEM INFORMASI MANAJEMEN BEASISWA TERINTEGRASI BERBASIS KELAYAKAN

Versi Dokumen	v1.0
Tanggal	01/06/2026
Semester	2025 Ganjil
Kelas	B
Kelompok	5
Mata Kuliah	Rekayasa Sistem Informasi
Dosen Pengampu	Bambang Widoyono
Universitas	Universitas Sebelas Maret

TECHNICAL SPECIFICATION DOCUMENT (TSD)**Riwayat Perubahan (Change History)**

Versi	Tanggal	Penulis	Status	Deskripsi Perubahan
v1.0	29/05/2026	Jimly	Draft	Versi awal dokumen TSD — Bagian 1, 2, dan 6
v1.1	30/05/2026	Kayla	Draft	Versi ke 2 dokumen TSD v1 - Bagian 5
v1.2	31/05/2026	Kayla	Draft	Versi ke 3 dokumen TSD v1 - Bagian 7
v1.3	31/05/2026	Nadhifa	Draft	Versi ke 4 dokumen TSD v1 - Bagian 3
v1.4	31/05/2026	Ibrahim	Draft	Versi ke 5 dokumen TSD v1 - Bagian 4
v2.0	01/06/2026	Nadhifa	Final	Versi final dokumen TSD

Anggota Tim & Peran

Nama	NIM	Peran	Tanggung Jawab Dokumen
Ibrahim Syauqi	L0224019	UI/UX Designer	TSD Bagian 4
Jimly Syahbatin	L0224033	Backend Dev	TSD Bagian 1, 2, 6
Nadhifa Sakha Tri Yasmin	L0224036	System Analyst	TSD Bagian 3
Kayla Maharani Muzaki	L0224036	DBA	TSD Bagian 5, 7

TSD Bagian 1 — System Architecture Specification

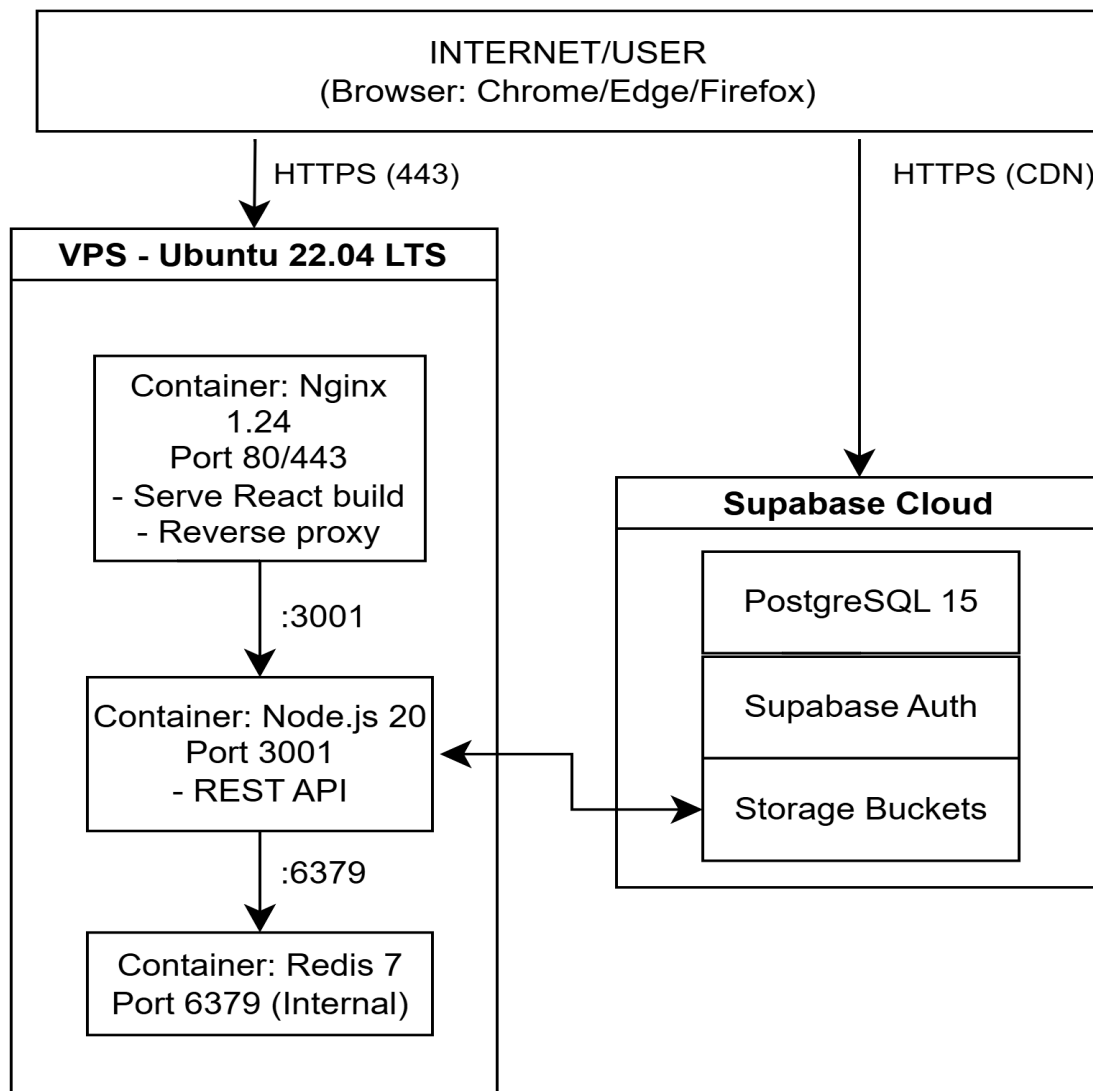
1.1 Technology Stack

Layer	Teknologi	Versi & Keterangan
Frontend	React.js 18 + Vite 5 + TailwindCSS 3	Single Page Application (SPA). Komponen berbasis React Hooks, state management dengan Zustand, routing dengan React Router v6. Build di-generate sebagai static files yang di-serve Nginx.
Backend	Node.js 20 LTS + TypeScript 5 + Express.js 4	RESTful API Server. Arsitektur berlapis: Router → Controller → Service → Repository. Semua logika bisnis diisolasi di Service class; Controller hanya mengurus HTTP request/response.
Database	Supabase (PostgreSQL 15)	RDBMS utama berbasis PostgreSQL yang dihosting oleh Supabase. Diakses via Supabase Client SDK (JavaScript) atau REST API. Mendukung Row Level Security (RLS) sebagai lapisan otorisasi tambahan di sisi database.
Cache / Session	Redis 7	Session store untuk JWT blacklist (token yang di-logout/revoke) dan cache hasil kalkulasi skor seleksi yang mahal. Diakses via library ioredis.
Object Storage	Supabase Storage	Penyimpanan berkas dokumen yang diunggah pengguna (foto, KTP, transkrip, dll).
Web Server	Nginx 1.24	Reverse proxy, TLS termination, dan static file server untuk build React. Mengatur CORS, rate limiting level jaringan, dan HSTS.
Runtime Container	Docker + Docker Compose	Seluruh komponen (Node.js API, Redis, Nginx)
OS / Infra	Ubuntu	Ubuntu 22.04 LTS
Autentikasi	Supabase Auth + JWT (HS256)	Supabase Auth mengelola lifecycle pengguna (registrasi, email verification, password reset). JWT diterbitkan oleh Supabase dengan klaim role kustom untuk penegakan RBAC di layer API.

Email	Supabase Auth Email (SMTP)	Pengiriman email transaksional: verifikasi akun, notifikasi status pengajuan, broadcast hasil seleksi, dan reminder laporan bulanan.
Report Generation	SheetJS (xlsx)	Generate file Excel (.xlsx) untuk ekspor data pengajuan dan laporan program beasiswa.
CI/CD	GitHub Actions	Pipeline otomatis: test → build Docker image → deploy ke staging/production via SSH. Deployment menggunakan rolling update tanpa downtime.

1.2 Deployment Architecture

a. Gambar



b. Keterangan

Komponen	Teknologi	Port	Catatan
API Gateway / Web Server	Nginx 1.24	80 / 443	HTTPS termination, HSTS header, redirect HTTP→HTTPS, static file serving untuk React build
Backend API	Node.js 20 + Express.js	3001	RESTful API server, dikemas dalam Docker container, diakses via Nginx reverse proxy
Database	Supabase (PostgreSQL 15)	5432 (internal)	Hosted di Supabase. Koneksi dari backend menggunakan Supabase Client SDK dengan service role key yang disimpan di environment variable
Cache / Session	Redis 7.2	6379	JWT blacklist, cache skor seleksi, distributed lock untuk cron job. Hanya dapat diakses dari jaringan internal Docker.
Frontend	React.js (static build)	Di-serve via Nginx	SPA — build ke static files

1.2.2 Pola Alur Traffic

- Browser pengguna → Nginx (port 443) → Backend API (port 3001) → Supabase DB
- Upload file: Browser → Nginx → Backend API → Supabase Storage (via SDK)
- Static assets React: Browser → Nginx → File system (build directory)
- Cron jobs (reminder & backup): Docker cron container → Backend API internal endpoint

1.3 Architectural Constraints

ID	Constraint (Batasan Arsitektur)	Referensi
AC-01	Frontend TIDAK boleh mengakses database atau Supabase langsung dari browser untuk operasi yang memerlukan privilege. Semua mutasi data harus melalui REST API backend.	Security — Mencegah eksposur Supabase service key di sisi klien.
AC-02	Business logic ditempatkan di Service class, BUKAN di Controller atau Router. Controller hanya bertanggung jawab atas parsing request, memanggil service, dan memformat response.	Clean Code / Separation of Concerns — Memudahkan unit testing dan pemeliharaan.
AC-03	Semua response API menggunakan envelope JSON standar: { status: 'success' 'error', data: any, message: string, errors?: object }.	Konsistensi kontrak API — Frontend dapat menangani response secara seragam.
AC-04	Semua endpoint yang mengubah data (POST, PATCH, PUT, DELETE) WAJIB memvalidasi JWT Bearer token di header Authorization.	Security — Mencegah akses tidak sah ke operasi tulis.
AC-05	Semua password pengguna harus di-hash menggunakan bcrypt dengan cost factor minimum 12 sebelum disimpan. Plaintext password TIDAK PERNAH disimpan atau di-log.	Security — Melindungi kredensial pengguna dari data breach.
AC-06	Setiap endpoint API harus memvalidasi role pengguna (RBAC) berdasarkan klaim di JWT token sebelum memproses request. Validasi role di frontend (UI hiding) BUKAN pengganti validasi API.	Security — Penegakan hak akses di lapisan yang tidak bisa di-bypass.
AC-07	Seluruh komunikasi client-server menggunakan HTTPS (TLS 1.2 minimum, TLS 1.3 diutamakan). HTTP murni diarahkan otomatis ke HTTPS via redirect 301.	Security / Compliance — Enkripsi data in-transit untuk semua pengguna.
AC-08	Semua tindakan kritis (login, logout, verifikasi dokumen, pengesahan seleksi, perubahan role, export data) WAJIB dicatat ke tabel audit_logs yang bersifat append-only. Tidak ada operasi UPDATE atau DELETE yang diizinkan pada tabel ini.	Accountability / Compliance — Jejak audit yang

		tidak dapat dimanipulasi.
AC-09	Upload file dibatasi maksimal 5 MB per file dengan format yang diizinkan: PDF dan PNG saja. Validasi dilakukan dua kali: di sisi klien (sebelum upload) dan di sisi server (setelah menerima file).	Security / Data Integrity — Mencegah upload file berbahaya dan menjaga konsistensi format.
AC-10	Konfigurasi bobot parameter seleksi (Akademik, Ekonomi, Prestasi, Dokumen) harus berjumlah tepat 100%. Sistem memblokir eksekusi seleksi jika total bobot tidak terpenuhi.	Business Rule Integrity — Menjamin keadilan dan konsistensi proses seleksi.
AC-11	Data finansial sensitif (nomor rekening bank) harus dienkrpsi menggunakan AES-256 sebelum disimpan di database. Tampilan di UI menggunakan data masking (hanya 4 digit terakhir yang terlihat).	Security / Privacy — Perlindungan data finansial pengguna sesuai regulasi perlindungan data.
AC-12	Satu akun siswa hanya dapat memiliki satu pengajuan beasiswa aktif pada satu waktu. Sistem memblokir pengajuan baru jika pengajuan sebelumnya belum selesai (status bukan DITOLAK atau selesai).	Business Rule Integrity — Mencegah duplikasi pengajuan dan menjaga integritas program beasiswa.

TSD Bagian 2 — Technology Stack & Environment Specification

2.1 Environment Comparison (Dev / Staging / Production)

Spesifikasi	Development	Staging	Production
Hardware / VM	Local machine developer (laptop/PC)	Local machine developer (laptop/PC)	VPS: 4 vCPU / 8 GB RAM / 100 GB SSD

	masing-masing anggota tim)	masing-masing anggota tim)	
OS	Windows 11	Windows 11	Ubuntu 22.04 LTS
Container Runtime	Docker Desktop + Docker Compose	Docker Desktop + Docker Compose	Docker Engine + Docker Compose
Web Server	Nginx (via Docker) atau Vite dev server untuk frontend	Nginx (via Docker) atau Vite dev server untuk frontend	Nginx 1.24
Backend Runtime	Node.js 20 LTS (via Docker)	Node.js 20 LTS (via Docker)	Node.js 20 LTS (via Docker)
Database	Supabase Cloud	Supabase Cloud	Supabase Cloud (project production)
Cache	Redis 7 (Docker)	Redis 7 (via Docker)	Redis 7 (via Docker)
Email	Supabase Auth Email (dev SMTP, tidak dikirim ke inbox nyata)	Supabase Auth Email (dev SMTP, tidak dikirim ke inbox nyata)	Supabase Auth Email (produksi, semua email terkirim)
URL	http://localhost:5173 (frontend) / http://localhost:3001 (API)	http://localhost:5173 (frontend) / http://localhost:3001 (API)	https://beasiswaku.id
SSL/TLS	Tidak ada (HTTP saja di localhost)	Tidak ada (HTTP saja di localhost)	Let's Encrypt via Certbot (auto-renew)
Environment Variables	.env.local (tidak di-commit ke repo)	.env.local (tidak di-commit ke repo)	.env.production (disimpan di server, di-inject saat deploy)
Tujuan	Coding & debug	Coding & debug	Live system

2.2 Supporting Tools

Kategori	Tool	Kegunaan dalam Proyek
API Testing	Postman	Membuat koleksi endpoint, menguji request/response, dan mendokumentasikan API secara visual.
DB Management	Supabase Studio	GUI bawaan Supabase untuk manajemen tabel, melihat data, menjalankan SQL query, dan memonitor storage.

API Docs	Swagger Editor	OpenAPI 3.0 YAML — sekaligus sebagai TSD Seksi 3 live
Version Control	Git + GitHub	Source control, pull request, code review. Branch strategy: main (production), dan feature/* branch.
CI/CD	GitHub Actions	Otomasi: lint → test → build Docker image → deploy ke production via SSH.
SSL	Certbot	Auto-renew Let's Encrypt SSL
Code Editor utama	VS Code	Ekstensi yang digunakan: ESLint, Prettier, TypeScript, Tailwind CSS IntelliSense, GitLens, Thunder Client
Unit & Integration Testing	Jest + Supertest	Unit test untuk service layer dan integration test untuk API endpoint Node.js/Express.
Container Management Lokal	Docker Desktop	Menjalankan seluruh stack (API, Redis, Nginx) secara lokal dalam environment yang terisolasi dan konsisten.
Remote Server Access	SSH + SCP	Akses dan manajemen VPS production. Deployment manual jika diperlukan di luar pipeline CI/CD.
SSL Certificate	Certbot	Otomasi penerbitan dan pembaruan sertifikat TLS dari Let's Encrypt untuk production.

TSD Bagian 3 — API Architecture & Endpoint Hierarchy

3.1 RESTful Standards

Seluruh komunikasi antara frontend dan backend menggunakan prinsip REST (Representational State Transfer) dengan konvensi berikut:

- Base URL: /api/v1/ (semua endpoint menggunakan prefix ini)
- Resources dalam bentuk plural noun: /users, /courses, /assignments
- Semua response menggunakan envelope JSON: { status, data, message, errors }
- Semua protected routes memerlukan JWT Bearer Token: Authorization: Bearer <token>
- HTTP Methods: GET (baca), POST (buat), PATCH (update parsial), DELETE (hapus)

3.2 Endpoint Index

Method	Auth	URL	Fungsi / Referensi FSD
Modul 1 : Autentikasi (Auth)			
POST	Public	POST /api/v1/auth/register	Registrasi akun siswa baru (FSD-2.1.1)
POST	Public	POST /api/v1/auth/login	Login & terima JWT token (FSD-2.1.3)
POST	JWT	POST /api/v1/auth/logout	Logout & blacklist token (FSD-2.1.3)
POST	Public	POST /api/v1/auth/forgot-password	Kirim email reset password (FSD-2.1.2)
POST	Public	POST /api/v1/auth/reset-password	Reset password via token email (FSD-2.1.2)
GET	JWT	GET /api/v1/auth/me	Ambil profil user yang sedang login (FSD-2.1.3)
Modul 2 : Manajemen User			
GET	Admin	GET /api/v1/users	List semua user (paginasi) (FSD-2.9.1)
GET	JWT	GET /api/v1/users/{id}	Detail user berdasarkan ID (FSD-2.9.1)
PATCH	JWT	PATCH /api/v1/users/{id}/profile	Update profil user (nama, telepon, dll) (FSD-2.2.1)

PATCH	Super Admin	PATCH /api/v1/users/{id}/role	Ubah role user (FSD-2.9.1)
PATCH	Super Admin	PATCH /api/v1/users/{id}/status	Aktivasi / nonaktifkan akun (FSD-2.9.1)
Modul 3 : Program Beasiswa			
GET	Public	GET /api/v1/programs	List program beasiswa tersedia (FSD-2.3.1)
GET	Public	GET /api/v1/programs/{id}	Detail program beasiswa (FSD-2.3.1)
POST	Admin	POST /api/v1/programs	Buat program beasiswa baru (FSD-2.9.2)
PATCH	Admin	PATCH /api/v1/programs/{id}	Edit program beasiswa (FSD-2.9.2)
PATCH	Admin	PATCH /api/v1/programs/{id}/status	Buka / tutup masa pendaftaran (FSD-2.3.1)
Modul 4 : Pengajuan Beasiswa (Applications)			
POST	JWT	POST /api/v1/applications	Pengajuan beasiswa (multi-step) (FSD-2.3.1)
GET	JWT	GET /api/v1/applications/my	Riwayat pengajuan siswa sendiri (FSD-2.3.4)
GET	Admin	GET /api/v1/applications	List semua pengajuan (admin) (FSD-2.4.2)
GET	JWT	GET /api/v1/applications/{id}	Detail pengajuan (FSD-2.4.2)
POST	JWT	POST /api/v1/applications/{id}/documents	Upload dokumen pendukung (FSD-2.3.2)
PATCH	Admin	PATCH /api/v1/applications/{id}/status	Update status pengajuan oleh admin (FSD-2.4.3)
Modul 5 : Seleksi & Penilaian			
POST	Admin	POST /api/v1/selections/{programId}/run	Jalankan proses seleksi otomatis (FSD-2.5.1)
GET	Admin	GET /api/v1/selections/{programId}/results	Hasil & ranking seleksi (FSD-2.5.1)

PATCH	Admin	PATCH /api/v1/selections/{programId}/weights	Set bobot parameter seleksi (FSD-2.5.1)
POST	Admin	POST /api/v1/selections/{programId}/finalize	Sahkan hasil seleksi (FSD-2.5.2)
Modul 6 : Laporan & Audit			
GET	Super Admin	GET /api/v1/admin/audit-logs	List audit log (paginasi, filter) (FSD-2.7.2)
GET	Admin	GET /api/v1/reports/programs/{id}	Laporan program beasiswa (FSD-2.8.1)
GET	Admin	GET /api/v1/reports/export	Export laporan ke Excel (.xlsx) (FSD-2.4.2)
GET	JWT	GET /api/v1/reports/monthly/{userId}	Laporan bulanan penerima beasiswa (FSD-2.6.2)
POST	JWT	POST /api/v1/reports/monthly	Upload laporan penggunaan dana bulanan (FSD-2.6.2)
PATCH	Admin	PATCH /api/v1/reports/monthly/{id}/verify	Verifikasi laporan dana bulanan (Admin) (FSD-2.4.4)
Modul 7 : Dashboard & Monitoring			
GET	Admin	GET /api/v1/admin/dashboard	Dashboard operasional Admin (statistik & chart) (FSD-2.4.1)
GET	Super Admin	GET /api/v1/admin/monitoring	Dashboard monitoring strategis Super Admin (FSD-2.7.1)
Modul 8 : Pencairan Dana			
POST	JWT	POST /api/v1/disbursements	Input data rekening pencairan dana (FSD-2.6.1)
GET	JWT	GET /api/v1/disbursements/my	Data rekening & status pencairan milik siswa (FSD-2.6.1)
PATCH	JWT	PATCH /api/v1/disbursements/{id}	Edit data rekening (jika belum diverifikasi) (FSD-2.6.1)
PATCH	Admin	PATCH /api/v1/disbursements/{id}/verify	Verifikasi & kunci data rekening oleh Admin (FSD-2.6.1)

Modul 9 : Sistem & Konfigurasi

GET	Super Admin	GET /api/v1/system/settings	Lihat konfigurasi aturan kelayakan global (FSD-2.9.2)
PATCH	Super Admin	PATCH /api/v1/system/settings	Update parameter IPK/penghasilan global (FSD-2.9.2)
POST	Super Admin	POST /api/v1/system/cron/reminder	Trigger manual auto-reminder laporan bulanan (FSD-2.8.3)
GET	Super Admin	GET /api/v1/users/{id}/bank-account	Lihat data rekening terenkripsi (masked) (FSD-2.9.3)

3.3 Endpoint Specification Card**1.a. POST /api/v1/auth/register**

Method	POST
URL	/api/v1/auth/register
Authentication	Tidak diperlukan (public endpoint)
Request Body (JSON)	{ full_name: string(required, min:3, max:100), email: string(required, email format, max:150), password: string(required, min:8, must contain uppercase + number + symbol), password_confirmation: string(required, must match password) }
Validasi Tambahan	1. Email belum terdaftar di tabel users. 2. Password strength: minimal 8 karakter, mengandung huruf besar, angka, dan simbol.
Response 201 (Sukses)	{ status: "success", data: { user: { id, full_name, email, role: "SISWA", is_active: true }, message: "Akun berhasil dibuat. Cek email untuk verifikasi." }, message: "Registrasi berhasil" }
Response 409 (Email Duplikat)	{ status: "error", error_code: "ERR-REG-01", message: "Email sudah terdaftar. Gunakan email lain atau login." }
Response 422 (Validasi Gagal)	{ status: "error", error_code: "ERR-VAL-01", message: "Data tidak valid", errors: { password: ["Password minimal 8 karakter dan mengandung huruf besar, angka, simbol"] } }
DB Tables Terpengaruh	users (CREATE) roles (READ — ambil role_id untuk SISWA)
DB Trigger / SP	trg_audit_users (AFTER INSERT on users) — mencatat event USER_CREATED ke audit_logs

Email Action	Kirim email verifikasi akun via Supabase Auth Email (SMTP)
Referensi FSD	FSD-2.1.1 — Registrasi Akun Siswa
Referensi SRS	FR-01
Error Codes	ERR-REG-05 (field kosong) ERR-REG-01 (format email) ERR-REG-06 (T&C belum dicentang) ERR-SRV-01 (internal error)

1.b. *POST /api/v1/auth/login*

Method	POST
URL	/api/v1/auth/login
Authentication	Tidak diperlukan (public endpoint)
Request Body (JSON)	{ email: string(required, email format), password: string(required, min:8) }
Rate Limiting	Maks. 5 percobaan gagal berturut-turut. Percobaan ke-5 mengunci akun sementara. IP-based + account-based (failed_login_count di tabel users).
Response 200 (Sukses)	{ status: "success", data: { token: "eyJ...", expires_in: 86400, user: { id, full_name, email, role } }, message: "Login berhasil" }
Response 401 (Kredensial Salah)	{ status: "error", error_code: "ERR-AUTH-02", message: "Email atau kata sandi tidak valid" }
Response 403 (Akun Terkunci)	{ status: "error", error_code: "ERR-AUTH-03", message: "Akun dikunci setelah 5 percobaan gagal." }
Response 403 (Masih Terkunci)	{ status: "error", error_code: "ERR-AUTH-04", message: "Akun masih dikunci. Coba lagi setelah locked_until." }
DB Tables Terpengaruh	users (READ, UPDATE last_login_at, failed_login_count, locked_until)
Cache (Redis)	Simpan JWT di Redis TTL 86400 detik. Counter gagal login TTL 900 detik.
DB Trigger	trg_audit_login — catat LOGIN_SUCCESS / LOGIN_FAILED + IP ke audit_logs
Referensi FSD	FSD-2.1.3 — Login Pengguna
Referensi SRS	FR-03

1.c. *POST /api/v1/auth/logout*

Method	POST
---------------	------

URL	/api/v1/auth/logout
Authentication	JWT Bearer Token (wajib)
Request Body	Tidak ada — token diambil dari Authorization header
Proses	1. Validasi token. 2. Ekstrak jti + exp. 3. Blacklist token di Redis: SET blacklist: {jti} '1' EX {sisat_ttl}. 4. Catat LOGOUT ke audit_logs.
Response 200 (Sukses)	{ status: "success", data: null, message: "Logout berhasil" }
Response 401 (Token Tidak Ada / Invalid)	{ status: "error", error_code: "ERR-AUTH-02", message: "Token tidak valid atau sudah kedaluwarsa" }
DB Tables Terpengaruh	users (READ — verifikasi identitas)
Cache (Redis)	INSERT ke blacklist Redis. Semua request berikutnya dengan token ini ditolak oleh middleware.
DB Trigger	trg_audit_login — catat action_type='LOGOUT' ke audit_logs
Referensi FSD	FSD-2.1.3 — Login Pengguna
Referensi SRS	FR-03

1.d. POST /api/v1/auth/forgot-password

Method	POST
URL	/api/v1/auth/forgot-password
Authentication	Tidak diperlukan (public endpoint)
Request Body (JSON)	{ email: string(required, email format) }
Catatan Keamanan	Response selalu 200 OK meskipun email tidak ditemukan — untuk mencegah user enumeration attack.
Response 200 (Sukses / Email Tidak Ditemukan)	{ status: "success", data: null, message: "Jika email terdaftar, tautan reset akan dikirimkan." }
Response 422 (Format Email Salah)	{ status: "error", error_code: "ERR-VAL-01", message: "Format email tidak valid.", errors: { email: ["Masukkan alamat email yang valid"] } }
Response 429 (Rate Limit)	{ status: "error", error_code: "ERR-RATE-01", message: "Terlalu banyak permintaan. Coba lagi dalam beberapa menit." }
DB Tables Terpengaruh	users (READ) password_reset_tokens (CREATE)
Email Action	Kirim link reset password via Supabase Auth Email. Token satu kali pakai, berlaku 1 jam.
Referensi FSD	FSD-2.1.2 — Verifikasi Email

Referensi SRS	FR-02
---------------	-------

1.e. *POST /api/v1/auth/reset-password*

Method	POST
URL	/api/v1/auth/reset-password
Authentication	Tidak diperlukan — diautentikasi via token dari email
Request Body (JSON)	{ token: string(required), password: string(required, min:8, harus mengandung huruf besar + angka + simbol), password_confirmation: string(required) }
Response 200 (Sukses)	{ status: "success", data: null, message: "Password berhasil direset. Silakan login dengan password baru." }
Response 400 (Token Kedaluwarsa / Tidak Valid)	{ status: "error", error_code: "ERR-VER-01", message: "Token reset tidak valid atau sudah kedaluwarsa." }
Response 422 (Password Tidak Memenuhi Syarat)	{ status: "error", error_code: "ERR-VAL-01", message: "Password tidak memenuhi syarat kekuatan.", errors: { password: ["Minimal 8 karakter, mengandung huruf besar, angka, dan simbol"] } }
DB Tables Terpengaruh	users (UPDATE password_hash) password_reset_tokens (DELETE — token diinvalidasi setelah pakai)
DB Trigger	trg_audit_users — catat PASSWORD_RESET ke audit_logs
Referensi FSD	FSD-2.1.2 — Verifikasi Email
Referensi SRS	FR-02

1.f. *GET /api/v1/auth/me*

Method	GET
URL	/api/v1/auth/me
Authentication	JWT Bearer Token (wajib)
Request Body	Tidak ada — identitas diambil dari klaim JWT
Response 200 (Sukses)	{ status: "success", data: { id, full_name, email, role, is_active, created_at, last_login_at, completion_pct }, message: "Profil berhasil diambil" }
Response 401 (Token Tidak Ada / Invalid)	{ status: "error", error_code: "ERR-AUTH-02", message: "Token tidak valid atau tidak ada" }
DB Tables Terpengaruh	users (READ) roles (READ via JOIN) profiles (READ)

DB View	v_users_with_roles — menyederhanakan JOIN users + roles
Referensi FSD	FSD-2.1.3 — Login Pengguna
Referensi SRS	FR-03

2.a. GET/api/v1/users

Method	GET
URL	/api/v1/users
Authentication	JWT Bearer Token Role: SuperAdmin
Query Parameters	?page=1 ?per_page=15 (max:100) ?search=keyword ?role=SISWA ADMIN SUPER_ADMIN ?is_active=true false ?sort_by=created_at full_name ?order=asc desc
Response 200 (Sukses)	{ status: "success", data: [{ id, full_name, email, role, is_active, created_at, last_login_at }], meta: { current_page, per_page, total, last_page }, message: "Data user berhasil diambil" }
Response 401 (Token Invalid)	{ status: "error", error_code: "ERR-AUTH-02", message: "Token tidak valid atau tidak ada" }
Response 403 (Role Tidak Cukup)	{ status: "error", error_code: "ERR-RBAC-01", message: "Akses ditolak. Hanya SuperAdmin yang dapat mengakses endpoint ini." }
DB Tables Terpengaruh	users (READ) roles (READ via JOIN)
DB View	v_users_with_roles
DB Trigger	trg_audit_access — catat ADMIN_PANEL_ACCESS ke audit_logs
Catatan	password_hash dan data sensitif TIDAK disertakan dalam response. Nomor rekening dimasking.
Referensi FSD	FSD-2.9.1 — Manajemen Pengguna & Hak Akses (RBAC)
Referensi SRS	FR-24

2.b. GET/api/v1/users/{id}

Method	GET
URL	/api/v1/users/{id}
Authentication	JWT Bearer Token Role: SuperAdmin (lihat semua) / Siswa (hanya milik sendiri)
Path Parameter	{id} — UUID user target

Response 200 (Sukses)	{ status: "success", data: { id, full_name, email, nim_nisn, phone, role, is_active, created_at, last_login_at, completion_pct }, message: "Data user berhasil diambil" }
Response 401 (Token Invalid)	{ status: "error", error_code: "ERR-AUTH-02", message: "Token tidak valid" }
Response 403 (Akses Ditolak)	{ status: "error", error_code: "ERR-RBAC-01", message: "Anda tidak berhak mengakses data user lain." }
Response 404 (Tidak Ditemukan)	{ status: "error", error_code: "ERR-USR-01", message: "User dengan ID tersebut tidak ditemukan." }
DB Tables Terpengaruh	users (READ) roles (READ via JOIN) profiles (READ)
DB View	v_users_with_roles
Referensi FSD	FSD-2.9.1 — Manajemen Pengguna & Hak Akses (RBAC)
Referensi SRS	FR-24

2.c. *PATCH/api/v1/users/{id}/profile*

Method	PATCH
URL	/api/v1/users/{id}/profile
Authentication	JWT Bearer Token Role: Siswa (milik sendiri) / Admin & SuperAdmin (semua)
Request Body (JSON)	{ full_name, phone, data_pribadi(jsonb), alamat(jsonb), ortu_wali(jsonb), akademik(jsonb) } — semua field opsional (PATCH parsial)
Response 200 (Sukses)	{ status: "success", data: { id, full_name, completion_pct }, message: "Profil berhasil diperbarui" }
Response 401 (Token Invalid)	{ status: "error", error_code: "ERR-AUTH-02", message: "Token tidak valid" }
Response 422 (Validasi Gagal)	{ status: "error", error_code: "ERR-PROF-02", message: "Data tidak valid.", errors: { "akademik.ipk": ["Nilai IPK harus antara 0.00 dan 4.00"] } }
Response 403 (Akses Ditolak)	{ status: "error", error_code: "ERR-RBAC-01", message: "Anda tidak berhak mengubah data user lain." }
DB Tables Terpengaruh	profiles (READ, UPDATE) users (READ)
DB Trigger	trg_audit_users — catat PROFILE_UPDATED ke audit_logs
Business Rule	BR-03: completion_pct dihitung ulang otomatis setelah update.
Referensi FSD	FSD-2.2.1 — Kelola Biodata Siswa

Referensi SRS	FR-04
---------------	-------

2.d. *PATCH/api/v1/users/{id}/role*

Method	PATCH
URL	/api/v1/users/{id}/role
Authentication	JWT Bearer Token Role: SuperAdmin ONLY
Path Parameter	{id} — UUID user target
Request Body (JSON)	{ role: string(required, enum: "SISWA" "ADMIN" "SUPER_ADMIN") }
Validasi Business Rule	1. SuperAdmin tidak dapat mengubah rolenya sendiri. 2. Tidak bisa menurunkan SuperAdmin terakhir. 3. Siswa dengan pengajuan aktif tidak bisa dijadikan ADMIN.
Response 200 (Sukses)	{ status: "success", data: { id, full_name, old_role: "SISWA", new_role: "ADMIN" }, message: "Role user berhasil diperbarui" }
Response 400 (Self-Change)	{ status: "error", error_code: "ERR-RBAC-02", message: "SuperAdmin tidak dapat mengubah role dirinya sendiri." }
Response 400 (Last SuperAdmin)	{ status: "error", error_code: "ERR-RBAC-03", message: "Tidak dapat mengubah role SuperAdmin terakhir." }
Response 403 (Role Tidak Cukup)	{ status: "error", error_code: "ERR-RBAC-01", message: "Akses ditolak. Hanya SuperAdmin yang dapat mengubah role." }
Response 404 (User Tidak Ditemukan)	{ status: "error", error_code: "ERR-USR-01", message: "User tidak ditemukan." }
DB Tables Terpengaruh	users (READ, UPDATE role_id) roles (READ) applications (READ — cek aktif)
DB Function	fn_check_last_superadmin — validasi tidak ada SuperAdmin tersisa
DB Trigger	trg_audit_users — catat ROLE_CHANGED + old_values + new_values ke audit_logs
Referensi FSD	FSD-2.9.1 — Manajemen Pengguna & Hak Akses (RBAC)
Referensi SRS	FR-24

2.e. *PATCH/api/v1/users/{id}/status*

Method	PATCH
URL	/api/v1/users/{id}/status

Authentication	JWT Bearer Token Role: SuperAdmin ONLY
Request Body (JSON)	{ is_active: boolean(required) }
Response 200 (Sukses)	{ status: "success", data: { id, full_name, is_active: false }, message: "Status akun berhasil diperbarui" }
Response 400 (Self-Deactivate)	{ status: "error", error_code: "ERR-RBAC-02", message: "SuperAdmin tidak dapat menonaktifkan akunnya sendiri." }
Response 403 (Role Tidak Cukup)	{ status: "error", error_code: "ERR-RBAC-01", message: "Akses ditolak." }
Response 404 (Tidak Ditemukan)	{ status: "error", error_code: "ERR-USR-01", message: "User tidak ditemukan." }
DB Tables Terpengaruh	users (READ, UPDATE is_active)
DB Trigger	trg_audit_users — catat USER_DEACTIVATED / USER_ACTIVATED ke audit_logs
Referensi FSD	FSD-2.9.1 — Manajemen Pengguna & Hak Akses (RBAC)
Referensi SRS	FR-24

3.a. GET/api/v1/programs

Method	GET
URL	/api/v1/programs
Authentication	Tidak diperlukan (public endpoint)
Query Parameters	?status=OPEN CLOSED DRAFT ?target_level=SMA PERGURUAN_TINGGI ?page=1 ?per_page=10
Response 200 (Sukses)	{ status: "success", data: [{ id, name, target_level, nominal, quota, deadline, status, description }], meta: { current_page, per_page, total, last_page }, message: "Data program berhasil diambil" }
Response 200 (Kosong)	{ status: "success", data: [], meta: { total: 0 }, message: "Belum ada program beasiswa yang tersedia." }
DB Tables Terpengaruh	scholarship_programs (READ)
Referensi FSD	FSD-2.3.1 — Pengajuan Beasiswa (Isi Formulir & Dokumen)
Referensi SRS	FR-05

3.b. GET/api/v1/programs/{id}

Method	GET
---------------	-----

URL	/api/v1/programs/{id}
Authentication	Tidak diperlukan (public endpoint)
Path Parameter	{id} — UUID program beasiswa
Response 200 (Sukses)	{ status: "success", data: { id, name, target_level, nominal, quota, deadline, status, description, requirements, created_at }, message: "Detail program berhasil diambil" }
Response 404 (Tidak Ditemukan)	{ status: "error", error_code: "ERR-PRG-01", message: "Program beasiswa tidak ditemukan." }
DB Tables Terpengaruh	scholarship_programs (READ)
Referensi FSD	FSD-2.3.1 — Pengajuan Beasiswa (Isi Formulir & Dokumen)
Referensi SRS	FR-05

3.c. *POST/api/v1/programs*

Method	POST
URL	/api/v1/programs
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Request Body (JSON)	{ name: string(required), target_level: enum(SMA PERGURUAN_TINGGI, required), nominal: integer(required, > 0), quota: integer(required, > 0), deadline: date(required, > today), description: string(optional), requirements: jsonb(optional) }
Response 201 (Sukses)	{ status: "success", data: { id, name, target_level, nominal, quota, deadline, status: "DRAFT" }, message: "Program beasiswa berhasil dibuat" }
Response 401 (Token Invalid)	{ status: "error", error_code: "ERR-AUTH-02", message: "Token tidak valid" }
Response 403 (Role Tidak Cukup)	{ status: "error", error_code: "ERR-RBAC-01", message: "Akses ditolak." }
Response 422 (Validasi Gagal)	{ status: "error", error_code: "ERR-VAL-01", message: "Data tidak valid.", errors: { deadline: ["Deadline harus setelah hari ini"] } }
DB Tables Terpengaruh	scholarship_programs (CREATE)
DB Trigger	trg_audit_programs — catat PROGRAM_CREATED ke audit_logs
Referensi FSD	FSD-2.9.2 — Konfigurasi Aturan Batas Kelayakan Beasiswa
Referensi SRS	FR-25

3.d. PATCH/api/v1/programs/{id}

Method	PATCH
URL	/api/v1/programs/{id}
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Request Body (JSON)	{ name, nominal, quota, deadline, description, requirements } — semua opsional (PATCH parsial)
Catatan Business Rule	Program berstatus OPEN atau CLOSED tidak dapat diedit field inti (nominal, quota). Hanya DRAFT yang bisa diedit bebas.
Response 200 (Sukses)	{ status: "success", data: { id, name, updated_at }, message: "Program berhasil diperbarui" }
Response 403 (Status Terkunci)	{ status: "error", error_code: "ERR-PRG-02", message: "Program aktif tidak dapat diedit. Tutup program terlebih dahulu." }
Response 404 (Tidak Ditemukan)	{ status: "error", error_code: "ERR-PRG-01", message: "Program tidak ditemukan." }
DB Tables Terpengaruh	scholarship_programs (READ, UPDATE)
DB Trigger	trg_audit_programs — catat PROGRAM_UPDATED ke audit_logs
Referensi FSD	FSD-2.9.2 — Konfigurasi Aturan Batas Kelayakan Beasiswa
Referensi SRS	FR-25

3.e. PATCH/api/v1/programs/{id}/status

Method	PATCH
URL	/api/v1/programs/{id}/status
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Request Body (JSON)	{ status: string(required, enum: "OPEN" "CLOSED" "DRAFT") }
Response 200 (Sukses)	{ status: "success", data: { id, name, status: "OPEN" }, message: "Status program berhasil diperbarui" }
Response 400 (Transisi Status Tidak Valid)	{ status: "error", error_code: "ERR-PRG-03", message: "Transisi status tidak valid. Program CLOSED tidak dapat dibuka ulang." }
Response 404 (Tidak Ditemukan)	{ status: "error", error_code: "ERR-PRG-01", message: "Program tidak ditemukan." }
DB Tables Terpengaruh	scholarship_programs (READ, UPDATE status)
DB Trigger	trg_audit_programs — catat STATUS_CHANGED ke audit_logs

Referensi FSD	FSD-2.3.1 — Pengajuan Beasiswa (Isi Formulir & Dokumen)
Referensi SRS	FR-05

4.a. *POST/api/v1/applications*

Method	POST
URL	/api/v1/applications
Authentication	JWT Bearer Token Role: Siswa
Request Body (JSON)	{ program_id: uuid(required), essay_motivasi: string(required, min:1000 karakter), data_akademik: jsonb(required — IPK/rata-rata nilai, institusi, prodi), data_ekonomi: jsonb(required — penghasilan ortu), prestasi_nonakademik: jsonb(optional) }
Validasi Business Rule	1. Profil harus 100% lengkap (BR-03). 2. Tidak ada pengajuan aktif lainnya (BR-01, BR-02). 3. Program berstatus OPEN dan kuota tersedia. 4. Essay minimal 1000 karakter (BR-06).
Response 201 (Sukses)	{ status: "success", data: { application_id, reference_no: "BEA-2026-XXXXXX", status: "PENDING" }, message: "Pengajuan berhasil dikirim" }
Response 409 (Sudah Ada Pengajuan Aktif)	{ status: "error", error_code: "ERR-APP-01", message: "Anda sudah memiliki pengajuan beasiswa yang sedang aktif." }
Response 400 (Profil Belum Lengkap)	{ status: "error", error_code: "ERR-PROF-01", message: "Profil belum 100% lengkap. Lengkapi biodata terlebih dahulu." }
Response 400 (Kuota Habis)	{ status: "error", error_code: "ERR-APP-02", message: "Kuota program ini telah habis." }
Response 422 (Essay Terlalu Pendek)	{ status: "error", error_code: "ERR-APP-04", message: "Essay motivasi minimal 1000 karakter.", errors: { essay_motivasi: ["Minimal 1000 karakter"] } }
DB Tables Terpengaruh	applications (CREATE) scholarship_programs (READ — cek kuota & status) profiles (READ — cek completeness)
DB SP	sp_submit_application — transaksi atomik: cek aktif + insert + update kuota
DB Trigger	trg_audit_applications — catat APPLICATION_SUBMITTED ke audit_logs
Email Action	Kirim email konfirmasi pengajuan ke siswa (BR-11)
Referensi FSD	FSD-2.3.1 — Pengajuan Beasiswa FSD-2.3.3 — Konfirmasi & Kirim
Referensi SRS	FR-05, FR-08

4.b. GET/api/v1/applications/my

Method	GET
URL	/api/v1/applications/my
Authentication	JWT Bearer Token Role: Siswa
Query Parameters	?page=1 ?per_page=10 ?sort_by=created_at ?order=desc
Response 200 (Ada Data)	{ status: "success", data: [{ application_id, reference_no, program_name, status, submitted_at, updated_at }], meta: { current_page, per_page, total, last_page }, message: "Riwayat pengajuan berhasil diambil" }
Response 200 (Belum Ada Pengajuan)	{ status: "success", data: [], meta: { total: 0 }, message: "Belum ada data pengajuan" }
Response 401 (Token Invalid)	{ status: "error", error_code: "ERR-AUTH-02", message: "Token tidak valid" }
DB Tables Terpengaruh	applications (READ) scholarship_programs (READ via JOIN)
DB View	v_application_summary — JOIN applications + programs + skor
Referensi FSD	FSD-2.3.4 — Pelacakan Status Pengajuan
Referensi SRS	FR-09

4.c. GET/api/v1/applications

Method	GET
URL	/api/v1/applications
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Query Parameters	?page=1 ?per_page=15 ?status=PENDING TERVERIFIKASI DITERIMA DITOLAK RE VISI ?program_id=uuid ?search=nama_siswa ?sort_by=submitted_at skor_total ?order=desc
Response 200 (Sukses)	{ status: "success", data: [{ application_id, reference_no, full_name, program_name, submitted_at, skor_total, status }], meta: { current_page, per_page, total, last_page }, message: "Data pengajuan berhasil diambil" }
Response 401 (Token Invalid)	{ status: "error", error_code: "ERR-AUTH-02", message: "Token tidak valid" }
Response 403 (Role Tidak Cukup)	{ status: "error", error_code: "ERR-RBAC-01", message: "Akses ditolak." }
DB Tables Terpengaruh	applications (READ) users (READ via JOIN) scholarship_programs (READ via JOIN)

DB View	v_application_summary
Referensi FSD	FSD-2.4.2 — Daftar & Detail Pengajuan
Referensi SRS	FR-13

4.d. GET/api/v1/applications/{id}

Method	GET
URL	/api/v1/applications/{id}
Authentication	JWT Bearer Token Role: Siswa (milik sendiri) / Admin / SuperAdmin
Path Parameter	{id} — UUID application
Response 200 (Sukses)	{ status: "success", data: { application_id, reference_no, status, essay_motivasi, data_akademik, data_ekonomi, documents: [{ type, file_path, is_valid }], history_logs, catatan_admin }, message: "Detail pengajuan berhasil diambil" }
Response 403 (Bukan Milik Sendiri)	{ status: "error", error_code: "ERR-RBAC-01", message: "Anda tidak berhak mengakses pengajuan ini." }
Response 404 (Tidak Ditemukan)	{ status: "error", error_code: "ERR-APP-07", message: "Pengajuan tidak ditemukan." }
DB Tables Terpengaruh	applications (READ) application_documents (READ) application_history (READ)
Referensi FSD	FSD-2.4.2 — Daftar & Detail Pengajuan FSD-2.3.4 — Pelacakan Status
Referensi SRS	FR-13, FR-09

4.e. POST/api/v1/applications/{id}/documents

Method	POST
URL	/api/v1/applications/{id}/documents
Authentication	JWT Bearer Token Role: Siswa (pemilik pengajuan)
Request Body	multipart/form-data: { document_type: enum(FOTO KTP KK TRANSKRIP SKTM SERTIFIKAT, required), file: binary(required) }
Validasi File	1. Ukuran maks 5 MB per file (BR-04). 2. Ekstensi hanya PDF dan PNG (BR-05). 3. Validasi dilakukan dua kali: client-side dan server-side.

Response 201 (Sukses)	{ status: "success", data: { document_id, document_type, file_path, file_size_kb, uploaded_at }, message: "Dokumen berhasil diunggah" }
Response 413 (File Terlalu Besar)	{ status: "error", error_code: "ERR-DOC-02", message: "Ukuran file melebihi batas 5 MB." }
Response 415 (Format Tidak Didukung)	{ status: "error", error_code: "ERR-DOC-01", message: "Format file tidak didukung. Hanya PDF dan PNG yang diizinkan." }
Response 409 (Dokumen Sudah Ada)	{ status: "error", error_code: "ERR-DOC-03", message: "Dokumen jenis ini sudah diunggah. Hapus yang lama terlebih dahulu." }
DB Tables Terpengaruh	application_documents (CREATE) applications (READ — verifikasi kepemilikan)
Object Storage	Simpan ke Supabase Storage Bucket 'documents/{application_id}/{document_type}' dengan RLS policy
Referensi FSD	FSD-2.3.2 — Formulir Pengajuan & Upload Dokumen
Referensi SRS	FR-06, FR-07

4.f. PATCH/api/v1/applications/{id}/status

Method	PATCH
URL	/api/v1/applications/{id}/status
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Request Body (JSON)	{ status: enum("TERVERIFIKASI" "REVISI" "DITOLAK", required), catatan_admin: string(required jika status REVISI atau DITOLAK, min:10) }
Validasi Business Rule	Catatan wajib diisi jika memilih REVISI atau DITOLAK (EF-1 FSD-2.4.3). Deteksi concurrency: cek updated_at sebelum eksekusi.
Response 200 (Sukses)	{ status: "success", data: { application_id, old_status: "PENDING", new_status: "TERVERIFIKASI" }, message: "Status pengajuan berhasil diperbarui" }
Response 400 (Catatan Kosong)	{ status: "error", error_code: "ERR-ACC-01", message: "Catatan alasan wajib diisi saat memilih Revisi atau Tolak." }
Response 409 (Concurrency Conflict)	{ status: "error", error_code: "ERR-CONC-01", message: "Data berubah oleh Admin lain. Muat ulang halaman." }
Response 404 (Tidak Ditemukan)	{ status: "error", error_code: "ERR-APP-07", message: "Pengajuan tidak ditemukan." }

DB Tables Terpengaruh	applications (READ, UPDATE status, catatan_admin) application_history (CREATE — log perubahan)
DB Trigger	trg_audit_applications — catat STATUS_CHANGED + old + new ke audit_logs
Email Action	Kirim notifikasi perubahan status ke email siswa (BR-14)
Referensi FSD	FSD-2.4.3 — Verifikasi Data Pendaftar
Referensi SRS	FR-14

5.a. POST/api/v1/selections/{programId}/run

Method	POST
URL	/api/v1/selections/{programId}/run
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Path Parameter	{programId} — UUID program beasiswa yang akan diseleksi
Request Body (JSON)	{ bobot_akademik: float(required), bobot_ekonomi: float(required), bobot_prestasi: float(required), bobot_dokumen: float(required) } — total keempat bobot HARUS = 100
Rumus Seleksi	Skor Total = (SA × WA) + (SE × WE) + (SP × WP) + (SD × WD). Hanya aplikasi berstatus TERVERIFIKASI yang diikutsertakan.
Response 200 (Sukses)	{ status: "success", data: { program_id, total_candidates: 42, ranking: [{ rank, application_id, full_name, skor_total, skor_akademik, skor_ekonomi, skor_prestasi, skor_dokumen }] }, message: "Kalkulasi skor selesai" }
Response 400 (Total Bobot != 100)	{ status: "error", error_code: "ERR-SEL-01", message: "Total bobot harus berjumlah tepat 100%." }
Response 400 (Tidak Ada Kandidat)	{ status: "error", error_code: "ERR-SEL-02", message: "Tidak ada pengajuan berstatus TERVERIFIKASI untuk program ini." }
Response 423 (Seleksi Sudah Final)	{ status: "error", error_code: "ERR-SEL-03", message: "Seleksi program ini sudah disahkan dan dikunci." }
DB Tables Terpengaruh	applications (READ, UPDATE skor_total) selection_results (CREATE/UPDATE)
Cache (Redis)	Simpan hasil ranking di Redis TTL 3600 detik untuk performa dashboard
DB Function	fn_calculate_score — kalkulasi skor per kandidat
Referensi FSD	FSD-2.5.1 — Hitung Skor Kelayakan
Referensi SRS	FR-15, FR-16

5.b. GET/api/v1/selections/{programId}/results

Method	GET
URL	/api/v1/selections/{programId}/results
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Query Parameters	?page=1 ?per_page=20 ?sort_by=rank skor_total ?order=asc
Response 200 (Sukses)	{ status: "success", data: { program_id, is_finalized: false, ranking: [{ rank, application_id, full_name, institusi, skor_total, status_rekomendasi }] }, meta: { current_page, total }, message: "Hasil seleksi berhasil diambil" }
Response 404 (Belum Ada Hasil)	{ status: "error", error_code: "ERR-SEL-02", message: "Belum ada hasil seleksi. Jalankan kalkulasi terlebih dahulu." }
Response 403 (Role Tidak Cukup)	{ status: "error", error_code: "ERR-RBAC-01", message: "Akses ditolak." }
DB Tables Terpengaruh	selection_results (READ) applications (READ via JOIN) users (READ via JOIN)
DB View	v_selection_ranking — JOIN selection_results + applications + users dengan computed rank
Referensi FSD	FSD-2.5.1 — Hitung Skor Kelayakan
Referensi SRS	FR-16

5.c. PATCH/api/v1/selections/{programId}/weights

Method	PATCH
URL	/api/v1/selections/{programId}/weights
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Request Body (JSON)	{ bobot_akademik: float, bobot_ekonomi: float, bobot_prestasi: float, bobot_dokumen: float } — total wajib = 100 (AC-10)
Response 200 (Sukses)	{ status: "success", data: { program_id, bobot_akademik, bobot_ekonomi, bobot_prestasi, bobot_dokumen }, message: "Bobot seleksi berhasil disimpan" }
Response 400 (Total Bobot != 100)	{ status: "error", error_code: "ERR-SEL-01", message: "Total keempat bobot harus berjumlah tepat 100%." }
Response 423 (Seleksi Sudah Final)	{ status: "error", error_code: "ERR-SEL-03", message: "Seleksi sudah disahkan. Bobot tidak dapat diubah." }
DB Tables Terpengaruh	selection_weights (READ, UPDATE) scholarship_programs (READ)

Referensi FSD	FSD-2.5.1 — Hitung Skor Kelayakan
Referensi SRS	FR-15

5.d. *POST/api/v1/selections/{programId}/finalize*

Method	POST
URL	/api/v1/selections/{programId}/finalize
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Request Body (JSON)	{ quota_diterima: integer(required, > 0), konfirmasi: boolean(required, harus true) }
Proses	1. Tandai pelamar dalam kuota → status DITERIMA. 2. Pelamar berikutnya → CADANGAN. 3. Selebihnya → DITOLAK. 4. Kunci seleksi (is_finalized=true). 5. Trigger broadcast email massal. 6. Buat antrian pencairan dana.
Response 200 (Sukses)	{ status: "success", data: { program_id, total_diterima, total_cadangan, total_ditolak, broadcast_queued: true }, message: "Hasil seleksi telah disahkan dan pengumuman sedang dikirim" }
Response 400 (Kuota Belum Diisi)	{ status: "error", error_code: "ERR-SEL-04", message: "Kuota penerima belum ditentukan." }
Response 423 (Sudah Disahkan)	{ status: "error", error_code: "ERR-SEL-03", message: "Hasil seleksi sudah disahkan dan bersifat final." }
DB Tables Terpengaruh	applications (UPDATE status massal) selection_results (UPDATE is_finalized) disbursement_queue (CREATE) failed_email_queue (CREATE)
DB Trigger	trg_audit_selections — catat SELECTION_FINALIZED + digital signature ke audit_logs
Email Action	Broadcast massal via background mailer. Retry 3x jika gagal (BR-24). Sisa masuk failed_email_queue.
Referensi FSD	FSD-2.5.2 — Sahkan Hasil Seleksi FSD-2.8.2 — Broadcast Pengumuman
Referensi SRS	FR-17, FR-22

6.a. *GET/api/v1/admin/audit-logs*

Method	GET
URL	/api/v1/admin/audit-logs
Authentication	JWT Bearer Token Role: SuperAdmin ONLY

Query Parameters	?page=1 ?per_page=25 ?action_type=LOGIN_SUCCESS ROLE_CHANGED ... ?user_id=uuid ?start_date=YYYY-MM-DD ?end_date=YYYY-MM-DD ?sort_by=created_at ?order=desc
Response 200 (Sukses)	{ status: "success", data: [{ log_id, user_email, action_type, resource_type, resource_id, old_values, new_values, ip_address, created_at }], meta: { current_page, per_page, total, last_page }, message: "Audit log berhasil diambil" }
Response 200 (Filter Tidak Ada Hasil)	{ status: "success", data: [], meta: { total: 0 }, message: "Tidak ada log yang cocok dengan filter tersebut." }
Response 403 (Role Tidak Cukup)	{ status: "error", error_code: "ERR-RBAC-01", message: "Akses ditolak. Hanya SuperAdmin." }
DB Tables Terpengaruh	audit_logs (READ — APPEND-ONLY, tidak ada UPDATE/DELETE)
DB View	v_audit_detail — JOIN audit_logs + users untuk tampilkan nama aktor
Referensi FSD	FSD-2.7.2 — Audit Log Sistem (Immutable)
Referensi SRS	FR-20

6.b. GET/api/v1/reports/programs/{id}

Method	GET
URL	/api/v1/reports/programs/{id}
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Path Parameter	{id} — UUID program beasiswa
Response 200 (Sukses)	{ status: "success", data: { program_id, program_name, total_pendaftar, total_diterima, acceptance_rate, avg_skor_total, sebaran_status: { PENDING, TERVERIFIKASI, DITERIMA, DITOLAK }, avg_ipk, avg_penghasilan_ortu }, message: "Laporan program berhasil diambil" }
Response 404 (Tidak Ditemukan)	{ status: "error", error_code: "ERR-PRG-01", message: "Program tidak ditemukan." }
Response 403 (Role Tidak Cukup)	{ status: "error", error_code: "ERR-RBAC-01", message: "Akses ditolak." }
DB Tables Terpengaruh	applications (READ) scholarship_programs (READ) profiles (READ via JOIN)
DB View	v_program_report — agregasi statistik per program
Referensi FSD	FSD-2.8.1 — Evaluasi Program Beasiswa

Referensi SRS	FR-21
---------------	-------

6.c. GET/api/v1/reports/export

Method	GET
URL	/api/v1/reports/export
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Query Parameters	?program_id=uuid(optional) ?status=enum(optional) ?start_date ?end_date ?format=xlsx (default)
Proses	Sistem mengambil data sesuai filter, generate file .xlsx menggunakan SheetJS, simpan sementara di server, kirim sebagai file download.
Response 200 (Sukses)	Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet Content-Disposition: attachment; filename=laporan_beasiswa_YYYYMMDD.xlsx [Binary file stream]
Response 408 (Timeout — Data > 500 Baris)	{ status: "error", error_code: "ERR-SYS-04", message: "Export timeout karena data terlalu besar. Gunakan filter untuk mempersempit rentang data." }
Response 403 (Role Tidak Cukup)	{ status: "error", error_code: "ERR-RBAC-01", message: "Akses ditolak." }
DB Tables Terpengaruh	applications (READ) users (READ) scholarship_programs (READ) application_documents (READ)
DB Trigger	trg_audit_access — catat DATA_EXPORTED ke audit_logs
Referensi FSD	FSD-2.4.2 — Daftar & Detail Pengajuan (fitur Export Excel)
Referensi SRS	FR-13

6.d. GET/api/v1/reports/monthly/{userID}

Method	GET
URL	/api/v1/reports/monthly/{userID}
Authentication	JWT Bearer Token Role: Siswa (milik sendiri) / Admin / SuperAdmin
Path Parameter	{userID} — UUID penerima beasiswa
Query Parameters	?page=1 ?per_page=12 ?year=2026

Response 200 (Ada Data)	{ status: "success", data: [{ report_id, period_month, period_year, total_amount, status, submitted_at, verified_at, expense_details }], meta: { current_page, total }, message: "Laporan bulanan berhasil diambil" }
Response 200 (Belum Ada Laporan)	{ status: "success", data: [], meta: { total: 0 }, message: "Belum ada laporan dana untuk user ini." }
Response 403 (Bukan Milik Sendiri)	{ status: "error", error_code: "ERR-RBAC-01", message: "Anda tidak berhak mengakses laporan user lain." }
DB Tables Terpengaruh	monthly_reports (READ) users (READ — verifikasi kepemilikan)
Referensi FSD	FSD-2.6.2 — Upload Laporan Penggunaan Dana FSD-2.4.4 — Verifikasi Laporan Dana Bulanan
Referensi SRS	FR-10, FR-18

6.e. *POST/api/v1/reports/monthly/*

Method	POST
URL	/api/v1/reports/monthly
Authentication	JWT Bearer Token Role: Siswa (penerima aktif)
Request Body (JSON)	{ period_month: int(required, 1-12), period_year: int(required), expense_details: jsonb(required — array { kategori, nominal }), total_amount: int(required), receipt_files: array<uuid>(required — ID file nota di storage) }
Validasi Business Rule	1. Tanggal kirim harus $\leq$ 25 bulan berjalan (BR-22). 2. Total nominal tidak boleh melebihi pagu dana bulan tersebut. 3. Laporan periode yang sama belum pernah dikirim. 4. Minimal satu file nota wajib dilampirkan.
Response 201 (Sukses)	{ status: "success", data: { report_id, period_month, period_year, status: "MENUNGGU_VERIFIKASI", submitted_at }, message: "Laporan berhasil dikirim dan menunggu verifikasi." }
Response 400 (Lewat Batas Tanggal 25)	{ status: "error", error_code: "ERR-LAP-01", message: "Batas waktu pengiriman laporan sudah ditutup (tanggal 25)." }
Response 400 (Nominal Melebihi Pagu)	{ status: "error", error_code: "ERR-LAP-04", message: "Total pengeluaran melebihi pagu dana bulanan." }
Response 409 (Sudah Pernah Kirim)	{ status: "error", error_code: "ERR-LAP-06", message: "Laporan untuk periode ini sudah pernah dikirim." }
Response 400 (Nota Belum Dilampirkan)	{ status: "error", error_code: "ERR-LAP-05", message: "Minimal satu bukti nota wajib dilampirkan." }

DB Tables Terpengaruh	monthly_reports (CREATE) disbursement_queue (READ — ambil pagu) applications (READ — validasi status aktif)
DB Trigger	trg_audit_reports — catat REPORT_SUBMITTED ke audit_logs
Referensi FSD	FSD-2.6.2 — Upload Laporan Penggunaan Dana
Referensi SRS	FR-10

6.f. PATCH/api/v1/reports/monthly/{id}/verify

Method	PATCH
URL	/api/v1/reports/monthly/{id}/verify
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Path Parameter	{id} — UUID monthly_report yang akan diverifikasi
Request Body (JSON)	{ verdict: enum("VALID" "REVISI", required), verification_notes: string(required jika verdict=REVISI, min:10) }
Validasi Business Rule	Catatan wajib diisi jika memilih REVISI (EF-3 FSD-2.4.4). Laporan berstatus VALID dikunci permanen — tidak dapat diubah kembali (BR-30).
Response 200 — Verdict VALID	{ status: "success", data: { report_id, status: "VALID", verified_at, verified_by }, message: "Laporan valid. Dana periode berikutnya masuk antrean pencairan." }
Response 200 — Verdict REVISI	{ status: "success", data: { report_id, status: "REVISI", verification_notes }, message: "Laporan dikembalikan ke siswa untuk diperbaiki." }
Response 423 (Sudah Valid/Terkunci)	{ status: "error", error_code: "ERR-LAP-07", message: "Laporan yang sudah disetujui Valid tidak dapat diubah." }
Response 400 (Catatan Kosong saat REVISI)	{ status: "error", error_code: "ERR-ACC-01", message: "Catatan alasan wajib diisi saat memilih Revisi." }
DB Tables Terpengaruh	monthly_reports (READ, UPDATE) disbursement_queue (CREATE — jika VALID, tambah antrean bulan berikutnya)
DB Trigger	trg_audit_reports — catat REPORT_VERIFIED ke audit_logs
Email Action	Notifikasi hasil verifikasi ke email siswa
Referensi FSD	FSD-2.4.4 — Verifikasi Laporan Dana Bulanan
Referensi SRS	FR-18

7.a. GET/api/v1/admin/dashboard

Method	GET
URL	/api/v1/admin/dashboard
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Query Parameters	?range=7 30 (rentang hari tren, default: 7)
Proses	Agregasi real-time: hitung total masuk, pending, terverifikasi, diterima. Muat data tren. Hitung sebaran status (donut chart). Data di-cache Redis 60 detik (BR-29).
Response 200 (Sukses)	{ status: "success", data: { count_stats: { total, pending, verified, accepted, rejected }, trend_data: [{ date, count }], donut_data: { PENDING: 35, TERVERIFIKASI: 48 }, last_updated }, message: "Dashboard berhasil dimuat" }
Response 200 (Data Kosong)	{ status: "success", data: { count_stats: { total: 0 }, trend_data: [], donut_data: {} }, message: "Belum ada data statistik." }
Response 403 (Role Tidak Cukup)	{ status: "error", error_code: "ERR-RBAC-01", message: "Akses ditolak." }
Response 504 (Timeout DB)	{ status: "error", error_code: "ERR-SYS-01", message: "Gagal memuat data dashboard. Coba lagi." }
DB Tables Terpengaruh	applications (READ — COUNT agregasi) scholarship_programs (READ)
Cache (Redis)	Cache hasil dengan key 'dashboard:admin' TTL 60 detik. Auto-refresh tiap 60 detik (BR-29).
Referensi FSD	FSD-2.4.1 — Dashboard Operasional Admin
Referensi SRS	FR-12

7.b. GET/api/v1/admin/monitoring

Method	GET
URL	/api/v1/admin/monitoring
Authentication	JWT Bearer Token Role: SuperAdmin ONLY
Query Parameters	?tahun_ajaran=2025 ?semester=ganjil genap
Proses	Query agregat KPI makro: Total Pendaftar, Total Penerima, Akumulasi Dana Cair, Success Rate. Muat data GeoJSON sebaran wilayah. Hitung top-5 institusi. Render tren semesteran.
Response 200 (Sukses)	{ status: "success", data: { kpi: { total_pendaftar, total_penerima, akumulasi_dana_cair, success_rate }, top_institusi: [{ nama, jumlah }], trend_semesteran: [{ semester, total }], map_data: { type: "FeatureCollection", features: [...] } }, message: "Monitoring strategis berhasil dimuat" }

Response 403 (Role Tidak Cukup)	{ status: "error", error_code: "ERR-ACCESS-01", message: "Akses ditolak. Hanya Super Admin." }
Response 504 (Analytics Timeout)	{ status: "error", error_code: "ERR-SYS-04", message: "Query analytics timeout. Coba sempitkan filter." }
Response 500 (GeoJSON Gagal)	{ status: "error", error_code: "ERR-MAP-01", message: "Gagal memuat peta. Modul peta disembunyikan sementara." }
DB Tables Terpengaruh	applications (READ) profiles (READ — institusi & wilayah) monthly_reports (READ — dana cair)
DB View	v_kpi_strategis — agregasi KPI per semester v_regional_stats — sebaran wilayah
Referensi FSD	FSD-2.7.1 — Dashboard Monitoring Strategis
Referensi SRS	FR-19

8.a. *POST/api/v1/disbursements*

Method	POST
URL	/api/v1/disbursements
Authentication	JWT Bearer Token Role: Siswa (status pengajuan DITERIMA)
Request Body (JSON)	{ bank_name: string(required), account_no: string(required, numerik 10-16 digit), account_holder: string(required), receipt_book_file: uuid(required), ktp_file: uuid(required) }
Enkripsi	account_no di-enkripsi AES-256 sebelum INSERT ke DB (BR-27). is_verified diset false.
Response 201 (Sukses)	{ status: "success", data: { disbursement_id, bank_name, account_no_masked: "XXXXXX5678", is_verified: false }, message: "Data rekening tersimpan dan menunggu verifikasi." }
Response 400 (Nomor Rekening Tidak Valid)	{ status: "error", error_code: "ERR-PROF-02", message: "Nomor rekening hanya boleh berisi angka." }
Response 409 (Data Sudah Ada)	{ status: "error", error_code: "ERR-BANK-02", message: "Data rekening sudah tersimpan. Gunakan endpoint edit." }
Response 403 (Bukan Penerima Aktif)	{ status: "error", error_code: "ERR-RBAC-01", message: "Hanya penerima beasiswa aktif yang dapat mengisi data pencairan." }
DB Tables Terpengaruh	disbursements (CREATE) applications (READ — validasi status DITERIMA)
Referensi FSD	FSD-2.6.1 — Input Data Pencairan Dana
Referensi SRS	FR-11

8.b. *GET/api/v1/disbursements/my*

Method	GET
URL	/api/v1/disbursements/my
Authentication	JWT Bearer Token Role: Siswa
Response 200 (Ada Data)	{ status: "success", data: { disbursement_id, bank_name, account_no_masked: "XXXXXX5678", account_holder, is_verified, verified_at, status_pencairan }, message: "Data pencairan berhasil diambil" }
Response 200 (Belum Ada)	{ status: "success", data: null, message: "Belum ada data rekening. Silakan lengkapi data pencairan." }
Response 401 (Token Invalid)	{ status: "error", error_code: "ERR-AUTH-02", message: "Token tidak valid." }
DB Tables Terpengaruh	disbursements (READ) disbursement_queue (READ — status pencairan)
Catatan Keamanan	account_no selalu dimasking. Dekripsi AES-256 di backend, tidak pernah keluar sebagai plaintext (AC-11, FSD-2.9.3).
Referensi FSD	FSD-2.6.1 — Input Data Pencairan Dana
Referensi SRS	FR-11

8.c. PATCH/api/v1/disbursements/{id}

Method	PATCH
URL	/api/v1/disbursements/{id}
Authentication	JWT Bearer Token Role: Siswa (pemilik)
Request Body (JSON)	{ bank_name, account_no, account_holder, receipt_book_file, ktp_file } — semua opsional
Validasi Business Rule	Data yang sudah diverifikasi (is_verified=true) tidak dapat diedit sama sekali (EF-3 FSD-2.6.1). Blokir di level UI dan API.
Response 200 (Sukses)	{ status: "success", data: { disbursement_id, bank_name, account_no_masked: "XXXXXX9999", is_verified: false }, message: "Data rekening berhasil diperbarui." }
Response 423 (Sudah Diverifikasi/Terkunci)	{ status: "error", error_code: "ERR-BANK-01", message: "Data rekening yang sudah diverifikasi tidak dapat diubah." }
Response 403 (Bukan Milik Sendiri)	{ status: "error", error_code: "ERR-RBAC-01", message: "Anda tidak berhak mengubah data ini." }
DB Tables Terpengaruh	disbursements (READ, UPDATE) — account_no di-enkripsi ulang AES-256 sebelum UPDATE

DB Trigger	trg_audit_disbursements — catat DISBURSEMENT_UPDATED ke audit_logs
Referensi FSD	FSD-2.6.1 — Input Data Pencairan Dana
Referensi SRS	FR-11

8.d. PATCH/api/v1/disbursements/{id}/verify

Method	PATCH
URL	/api/v1/disbursements/{id}/verify
Authentication	JWT Bearer Token Role: Admin atau SuperAdmin
Request Body (JSON)	{ is_verified: boolean(required), catatan: string(optional) }
Response 200 (Diverifikasi)	{ status: "success", data: { disbursement_id, is_verified: true, verified_at, verified_by }, message: "Data rekening telah diverifikasi dan dikunci." }
Response 200 (Ditolak)	{ status: "success", data: { disbursement_id, is_verified: false, catatan }, message: "Data rekening ditolak. Siswa diminta melengkapi ulang." }
Response 404 (Tidak Ditemukan)	{ status: "error", error_code: "ERR-BANK-03", message: "Data rekening tidak ditemukan." }
DB Tables Terpengaruh	disbursements (READ, UPDATE is_verified, verified_at, verified_by)
DB Trigger	trg_audit_disbursements — catat DISBURSEMENT_VERIFIED ke audit_logs
Email Action	Notifikasi status verifikasi rekening ke email siswa
Referensi FSD	FSD-2.6.1 — Input Data Pencairan Dana
Referensi SRS	FR-11

9.a. GET/api/v1/system/settings

Method	GET
URL	/api/v1/system/settings
Authentication	JWT Bearer Token Role: SuperAdmin ONLY
Response 200 (Sukses)	{ status: "success", data: { min_ipk_sma: null, min_ipk_pt: 2.75, max_penghasilan_ortu: 3000000, updated_by: { id, full_name }, updated_at }, message: "Konfigurasi sistem berhasil diambil" }
Response 403 (Role Tidak Cukup)	{ status: "error", error_code: "ERR-RBAC-01", message: "Akses ditolak. Hanya Super Admin." }

DB Tables Terpengaruh	system_settings (READ)
Referensi FSD	FSD-2.9.2 — Konfigurasi Aturan Batas Kelayakan Beasiswa
Referensi SRS	FR-25

9.b. PATCH/api/v1/system/settings

Method	PATCH
URL	/api/v1/system/settings
Authentication	JWT Bearer Token Role: SuperAdmin ONLY
Request Body (JSON)	{ min_ipk_sma: float(optional, 0.00–4.00), min_ipk_pt: float(optional, 0.00–4.00), max_penghasilan_ortu: int(optional, > 0) }
Validasi Business Rule	1. IPK harus dalam rentang 0.00–4.00 (EF-1). 2. Tidak boleh berisi teks non-numerik (EF-2). 3. Perubahan bersifat non-retroaktif — tidak mempengaruhi pengajuan yang sudah terkirim (BR-32).
Response 200 (Sukses)	{ status: "success", data: { min_ipk_pt: 3.00, max_penghasilan_ortu: 2500000, updated_by, updated_at }, message: "Konfigurasi aturan kelayakan berhasil diperbarui." }
Response 422 (IPK di Luar Rentang)	{ status: "error", error_code: "ERR-PROF-02", message: "Nilai IPK tidak valid.", errors: { min_ipk_pt: ["Harus antara 0.00 dan 4.00"] } }
Response 422 (Format Non-Numerik)	{ status: "error", error_code: "ERR-PROF-02", message: "Parameter harus berupa nilai numerik valid." }
Response 423 (DB Deadlock)	{ status: "error", error_code: "ERR-SYS-01", message: "Gagal menyimpan konfigurasi. Coba beberapa saat lagi." }
DB Tables Terpengaruh	system_settings (READ, UPDATE)
DB Trigger	trg_audit_settings — catat SETTINGS_UPDATED + old_values + new_values ke audit_logs
Referensi FSD	FSD-2.9.2 — Konfigurasi Aturan Batas Kelayakan Beasiswa
Referensi SRS	FR-25

9.c. POST/api/v1/system/cron/reminder

Method	POST
URL	/api/v1/system/cron/reminder
Authentication	JWT Bearer Token Role: SuperAdmin ONLY

Request Body	Tidak ada — langsung trigger eksekusi cron job reminder
Catatan	Dalam kondisi normal, cron job berjalan otomatis tanggal 20 pukul 00:00 WIB. Endpoint ini untuk trigger manual oleh Super Admin. Redis distributed lock menjamin tidak ada pengiriman ganda (EF-3 FSD-2.8.3).
Response 200 (Sukses)	{ status: "success", data: { recipients_targeted: 42, emails_queued: 42, skipped_already_reported: 5 }, message: "Auto-reminder berhasil dijadwalkan ke antrean pengiriman." }
Response 200 (Tidak Ada Target)	{ status: "success", data: { recipients_targeted: 0 }, message: "Semua penerima aktif sudah mengirim laporan bulan ini." }
Response 423 (Job Sedang Berjalan)	{ status: "error", error_code: "ERR-CRON-01", message: "Cron job sedang berjalan. Tunggu hingga selesai." }
DB Tables Terpengaruh	applications (READ) monthly_reports (READ — cek belum lapor) users (READ — ambil email)
Cache (Redis)	Distributed lock 'cron:reminder:lock' TTL 300 detik untuk cegah double-run
DB Trigger	trg_audit_system — catat CRON_TRIGGERED ke audit_logs
Referensi FSD	FSD-2.8.3 — Auto-Reminder Batas Waktu Pelaporan Bulanan
Referensi SRS	FR-23

9.d. GET/api/v1/users/{id}/bank-account

Method	GET
URL	/api/v1/users/{id}/bank-account
Authentication	JWT Bearer Token Role: Siswa (milik sendiri) / Admin / SuperAdmin
Path Parameter	{id} — UUID user pemilik rekening
Response 200 (Sukses)	{ status: "success", data: { disbursement_id, bank_name, account_no_masked: "XXXXXX5678", account_holder, is_verified, verified_at }, message: "Data rekening berhasil diambil" }
Response 200 (Belum Ada)	{ status: "success", data: null, message: "Belum ada data rekening tersimpan." }
Response 403 (Bukan Milik Sendiri)	{ status: "error", error_code: "ERR-RBAC-01", message: "Anda tidak berhak mengakses data rekening user lain." }
Response 500 (Kunci Dekripsi Error)	{ status: "error", error_code: "ERR-SYS-01", message: "Gagal membaca data. Hubungi administrator sistem." }

DB Tables Terpengaruh	disbursements (READ) — dekripsi AES-256 di backend, masking sebelum kirim ke frontend
DB Trigger	trg_audit_access — catat SENSITIVE_DATA_READ ke audit_logs (AC-11)
Catatan Keamanan	Plaintext account_no TIDAK PERNAH keluar dari layer backend. Hanya masked string yang dikirim ke response (FSD-2.9.3).
Referensi FSD	FSD-2.9.3 — Pengamanan Enkripsi Data Sensitif (Data Masking)
Referensi SRS	FR-26

3.4 Appendix — API → DB Impact Mapping

Rubrik RPS: 'Mapping DB impact: Impact Table, SP, function, view, index, trigger terhadap API dan fungsionalitas' — Tabel ini WAJIB dilengkapi.

API Endpoint	Tables (CRUD)	Stored Proc	Views	Functions	Triggers
POST /auth/register	users(C) roles(R)	—	—	—	trg_audit_users
POST /auth/login	users(R,U) failed_attempts (R,C,D)	—	—	fn_check_lockout	trg_audit_login
POST /auth/logout	users(R)	—	—	—	trg_audit_login
POST /auth/forgot-password	users(R) pwd_reset_tokens(C)	—	—	—	—
POST /auth/reset-password	users(U) pwd_reset_tokens(D)	—	—	—	trg_audit_users
GET /auth/me	users(R) roles(R)	—	v_users_with_roles	—	—
GET /users	users(R) roles(R)	—	v_users_with_roles	—	trg_audit_access

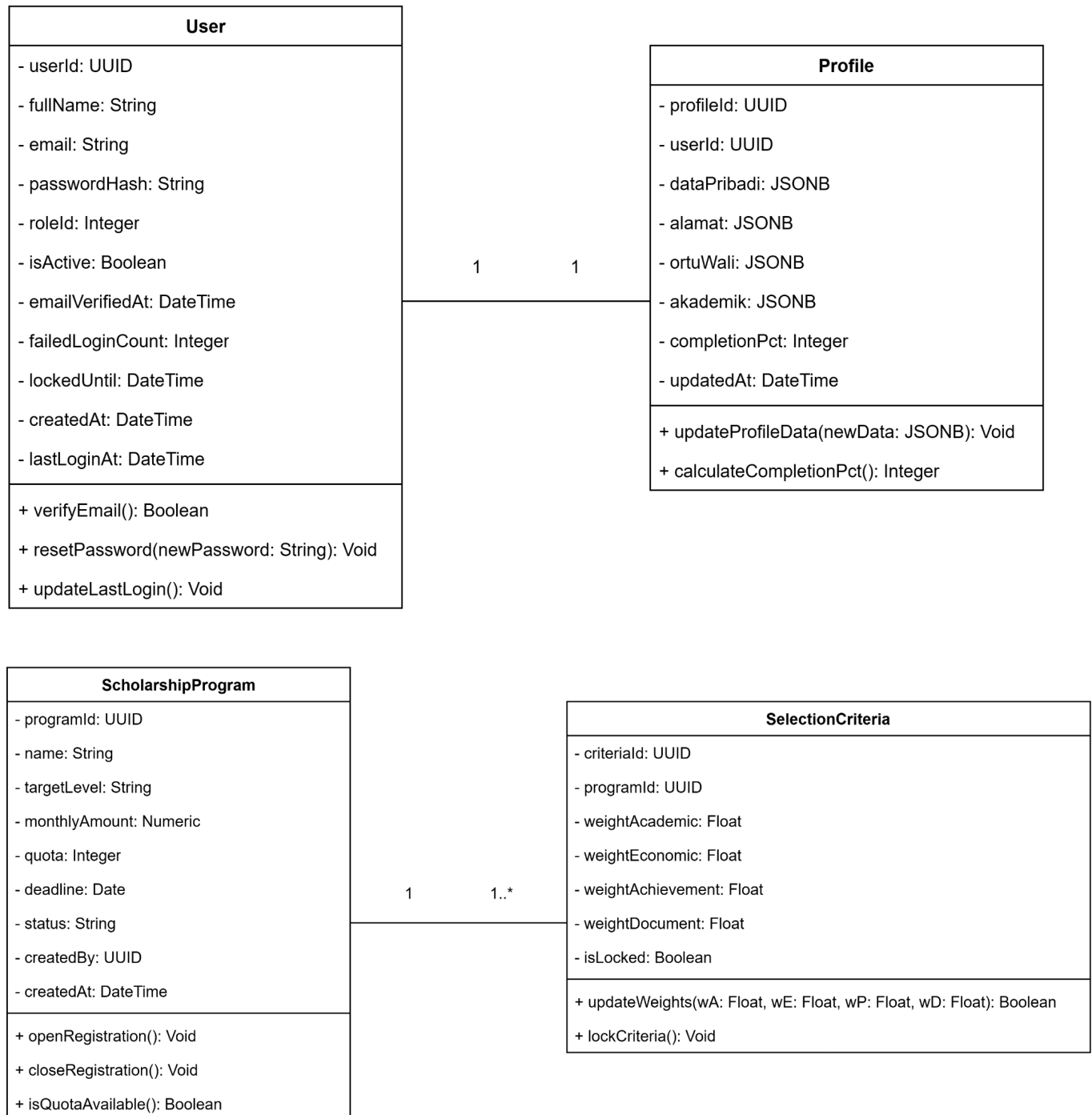
GET /users/{id}	users(R) roles(R)	—	v_users_with_roles	—	—
PATCH /users/{id}/profile	profiles(R,U) users(R)	—	—	—	trg_audit_users
PATCH /users/{id}/role	users(R,U) roles(R) applications(R)	—	—	fn_check_last_superuseradmin	trg_audit_users
PATCH /users/{id}/status	users(R,U)	—	—	—	trg_audit_users

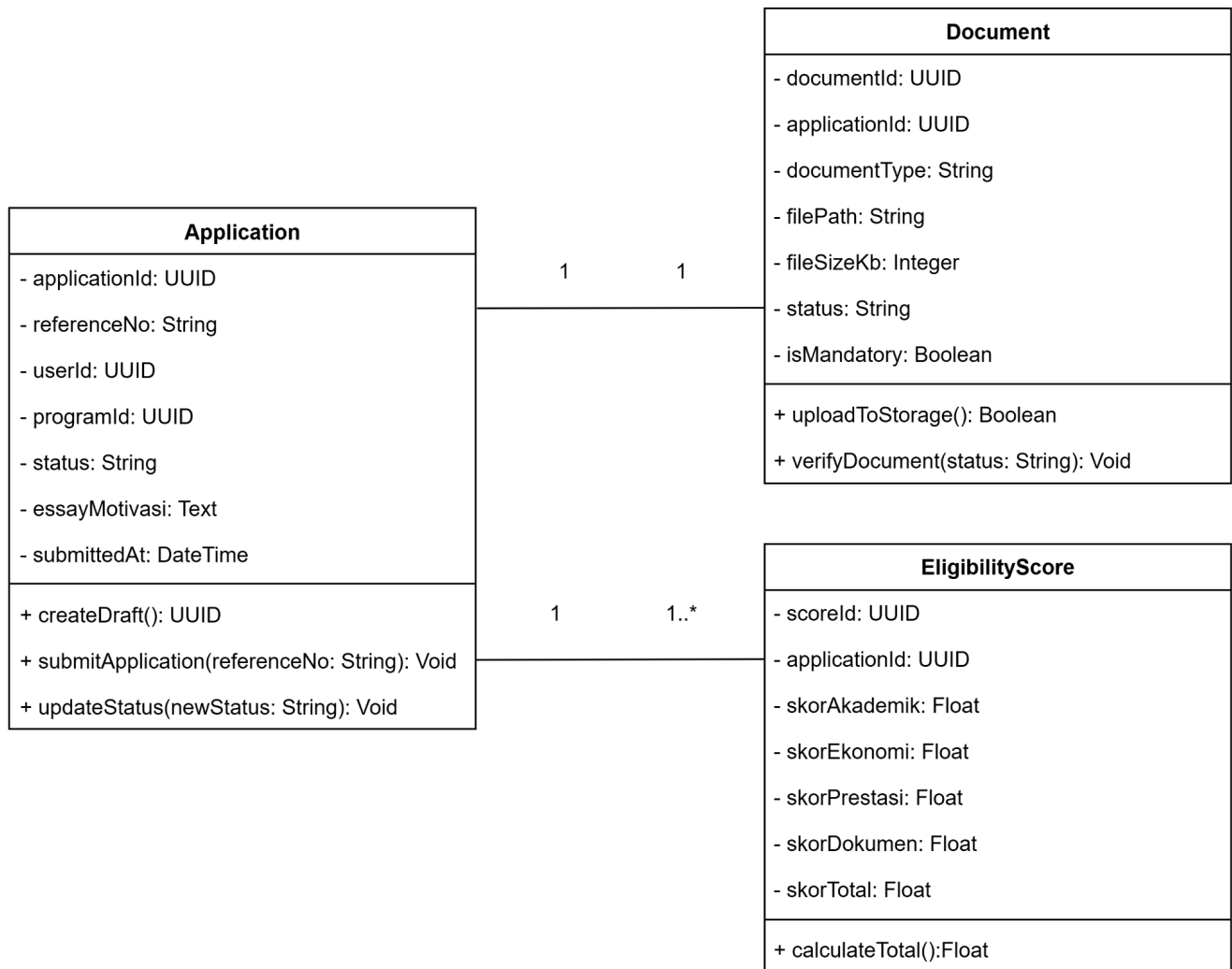
Legenda: C=CREATE R=READ U=UPDATE D=DELETE | Highlighted row enroll = contoh operasi DB paling kompleks (SP + function + trigger untuk atomisitas)

TSD Bagian 4 — Database Design Specification

4.1 Class Diagram

CORE MODELS / ENTITIES CLASS DIAGRAM





SERVICE / LOGIC CLASSES (BUSINESS PROCESS DIAGRAM)

AuthService
+ registerStudent(fullName: String, email: String, ...): User + login(email: String, passwordPlain: String): JWTTokenEnvelope + logout(userId: UUID): Boolean - generateJWTToken(user: User): JWTTokenEnvelope - handleAccountLockout(user: User): Void

VerificationService
+ reviewDocument(documentId: UUID, status: String, notes: String):Void + finalizeVerification(applicationId: UUID, decision: String): Void

SelectionEngine
+ runAutomatedSelection(programId: UUID, criteriaId: UUID): List<Rank> + finalizeSelectionResult(programId: UUID): Boolean - calculateScore(app: Application, cr: SelectionCriteria): Float

FundReportService
+ submitMonthlyReport(userId: UUID, expenseData: JSONB): FundReport + verifyFinancialReport(reportId: UUID, isValid: Boolean): Void - triggerNextDisbursement(userId: UUID): Void

4.2 Table Definition

a. table 1- users

Kolom	Tipe Data	Null?	Default	Constraint / Keterangan
user_id	UUID	NOT NULL	gen_random_uuid()	PRIMARY KEY — Pengidentifikasi unik setiap akun.
full_name	VARCHAR(100)	NOT NULL	—	Nama lengkap pemilik akun.
email	VARCHAR(150)	NOT NULL	—	UNIQUE — Alamat email aktif pendaftar.
password_hash	VARCHAR(255)	NOT NULL	—	Hasil enkripsi kata sandi menggunakan bcrypt cost=12.
role_id	INTEGER	NOT NULL	—	FK → roles(role_id) ON DELETE RESTRICT.
is_active	BOOLEAN	NOT NULL	TRUE	Status aktif akun (<i>Soft delete flag</i>).
email_verified_at	TIMESTAMP	NULLABLE	NULL	Waktu verifikasi email pendaftar setelah registrasi.
failed_login_count	INTEGER	NOT NULL	0	Menghitung gagal login berturut-turut (Maksimal 5x).

locked_until	TIMESTAMP	NULLABLE	NULL	Waktu pembukaan kunci akun jika terblokir sementara otomatis (30 menit).
created_at	TIMESTAMP	NOT NULL	NOW()	Penanda waktu pendaftaran akun dibuat.
last_login_at	TIMESTAMP	NULLABLE	NULL	Jejak waktu login terakhir yang berhasil.

b. table 2- profiles

Kolom	Tipe Data	Null?	Default	Constraint / Keterangan
profile_id	UUID	NOT NULL	gen_random_uuid()	PRIMARY KEY — ID rekam profil siswa.
user_id	UUID	NOT NULL	—	FK → users(user_id) ON DELETE CASCADE (Relasi 1:1 sesuai Class Diagram).
data_pribadi	JSONB	NOT NULL	—	Payload data fleksibel isi komponen biodata dasar siswa.
alamat	JSONB	NOT NULL	—	Payload data detail alamat domisili pendaftar.

ortu_wali	JSONB	NOT NULL	—	Payload data latar belakang orang tua atau wali pendaftar.
akademik	JSONB	NOT NULL	—	Payload data capaian prestasi akademik/rapor sekolah.
completion_pct	INTEGER	NOT NULL	0	Persentase kelengkapan isi profil (\$0\%\$ hingga \$100\%\$).
status_pengisian	VARCHAR(30)	NOT NULL	'DRAFT'	Berisi status simpanan: 'DRAFT' atau 'PERMANENT'.
updated_at	TIMESTAMP	NOT NULL	NOW()	Catatan waktu pembaruan data terakhir.

c. table 3- scholarships

Kolom	Tipe Data	Null?	Default	Constraint / Keterangan
program_id	UUID	NOT NULL	gen_random_uuid()	PRIMARY KEY — ID unik program master beasiswa.
name	VARCHAR(100)	NOT NULL	—	Nama beasiswa (Contoh: 'Beasiswa SMA', 'Beasiswa Kuliah').
target_level	VARCHAR(30)	NOT NULL	—	Batasan sasaran jenjang pendaftar: 'SMA' atau 'PERGURUAN_TINGGI'.

monthly_amount	NUMERIC(12,2)	NOT NULL	—	Besaran bantuan pendanaan bulanan (SMA: 750rb, PT: 1jt).
quota	INTEGER	NOT NULL	—	Batas maksimum kuota jumlah siswa penerima yang ditampung.
deadline	DATE	NOT NULL	—	Batas akhir waktu pengajuan berkas ditutup otomatis.
status	VARCHAR(20)	NOT NULL	'OPEN'	Status aktivitas program, berisi: 'OPEN' atau 'CLOSED'.
created_by	UUID	NOT NULL	—	FK → users(user_id) (Aktor admin pembuat program).
created_at	TIMESTAMP	NOT NULL	NOW()	Catatan waktu pembukaan program master beasiswa.

d. table 4- criteria

Kolom	Tipe Data	Null?	Default	Constraint / Keterangan
criteria_id	UUID	NOT NULL	gen_random_uuid()	PRIMARY KEY — ID pengaturan parameter hitung.
program_id	UUID	NOT NULL	—	FK → scholarships(program_id) ON DELETE CASCADE.
weight_academic	NUMERIC(5,2)	NOT NULL	40.00	Bobot nilai rapor/IPK siswa dalam persentase.
weight_economic	NUMERIC(5,2)	NOT NULL	35.00	Bobot kondisi keterbatasan finansial keluarga.

weight_achievement	NUMERIC(5, 2)	NOT NULL	15.00	Bobot poin piagam prestasi non-akademik penunjang.
weight_document	NUMERIC(5, 2)	NOT NULL	10.00	Bobot kualitas keabsahan pemenuhan berkas fisik.
is_locked	BOOLEAN	NOT NULL	FALSE	Status kunci konfigurasi jika hasil pemeringkatan disahkan.

e. table 5- applications

Kolom	Tipe Data	Null?	Default	Constraint / Keterangan
application_id	UUID	NOT NULL	gen_random_uuid()	PRIMARY KEY — ID induk pengajuan berkas seleksi.
reference_no	VARCHAR(50)	NOT NULL	—	UNIQUE — Nomor unik registrasi pendaftaran otomatis.
user_id	UUID	NOT NULL	—	FK → users(user_id) ON DELETE RESTRICT.
program_id	UUID	NOT NULL	—	FK → scholarships(program_id) ON DELETE RESTRICT.
status	VARCHAR(30)	NOT NULL	'PROSES'	Status seleksi: 'PROSES', 'TERIMA', atau 'TOLAK'.
essay_motivasi	TEXT	NOT NULL	—	Teks esai deskripsi komitmen diri pendaftar beasiswa.
submitted_at	TIMESTAMP	NOT NULL	NOW()	Tanggal dan jam berkas dikirimkan secara permanen.

f. table 6- documents

Kolom	Tipe Data	Null?	Default	Constraint / Keterangan
document_id	UUID	NOT NULL	gen_random_uuid()	PRIMARY KEY — ID rekam berkas arsip.
application_id	UUID	NOT NULL	—	FK → applications(application_id) ON DELETE CASCADE.
document_type	VARCHAR(50)	NOT NULL	—	Kategori berkas (Contoh: 'KTP', 'KARTU_KELUARGA', 'TRANSKRIP').
file_path	VARCHAR(255)	NOT NULL	—	Jalur alamat lokasi berkas disimpan di server cloud/storage.
file_size_kb	INTEGER	NOT NULL	—	Ukuran file dalam satuan Kilobytes untuk validasi batas berkas.
status	VARCHAR(30)	NOT NULL	'PENDING'	Status verifikasi berkas: 'PENDING', 'VALID', atau 'REJECTED'.
is_mandatory	BOOLEAN	NOT NULL	TRUE	Penanda dokumen wajib diisi/diunggah atau tidak.

g. table 7- eligibility_scores

Kolom	Tipe Data	Null?	Default	Constraint / Keterangan
score_id	UUID	NOT NULL	gen_random_uuid()	PRIMARY KEY — ID rekaman skor kelayakan.
application_id	UUID	NOT NULL	—	FK → applications(application_id) ON DELETE CASCADE.
skor_akademik	NUMERIC(5,2)	NOT NULL	0.00	Hasil perkalian nilai akademik dengan bobot kriteria.

skor_ekonomi	NUMERIC(5,2)	NOT NULL	0.00	Hasil perkalian kondisi ekonomi dengan bobot kriteria.
skor_prestasi	NUMERIC(5,2)	NOT NULL	0.00	Hasil perkalian piagam prestasi dengan bobot kriteria.
skor_dokumen	NUMERIC(5,2)	NOT NULL	0.00	Hasil perkalian kelengkapan berkas dengan bobot kriteria.
skor_total	NUMERIC(5,2)	NOT NULL	0.00	Total nilai akhir (\$Sum\$) komponen sebagai dasar peringkat kelulusan.

h. table 8- roles

Kolom	Tipe Data	Null?	Default	Constraint / Keterangan
role_id	INTEGER	NOT NULL	—	PRIMARY KEY — ID tingkat peran pengguna.
role_name	VARCHAR(30)	NOT NULL	—	UNIQUE — Berisi 'SISWA', 'ADMIN', atau 'SUPER_ADMIN'.
description	VARCHAR(255)	NULLABLE	NULL	Keterangan cakupan wewenang peran tersebut.

h. table 8- menus

Kolom	Tipe Data	Null?	Default	Constraint / Keterangan
menu_id	UUID	NOT NULL	gen_random_uuid()	PRIMARY KEY — ID komponen menu sidebar.

role_id	INTEGER	NOT NULL	—	FK → roles(role_id) ON DELETE CASCADE.
title	VARCHAR(50)	NOT NULL	—	Label tekstual nama menu yang muncul di UI.
route_path	VARCHAR(100)	NOT NULL	—	Alamat path URL internal halaman React frontend.
icon_name	VARCHAR(50)	NULLABLE	NULL	Nama kode ikon grafis yang dirender di UI.
sort_order	INTEGER	NOT NULL	0	Urutan susunan letak menu dari atas ke bawah.



TSD Bagian 5 — Stored Procedures, DML & DB Objects

Bagian ini memenuhi rubrik RPS: 'Arsitektur API: SP, DML DB terkait' & 'Mapping DB impact: Table, SP, function, view, index, trigger terhadap API dan fungsionalitas'

5.1 Stored Procedures (SP)

sp_submit_application

Deskripsi singkat: Tujuannya mengamankan proses pengajuan beasiswa dengan transaksi atomik guna mencegah race condition (duplikasi pengajuan aktif) sekaligus menegakkan BR-01 (hanya satu pengajuan aktif) dan BR-03 (profil 100% lengkap).

Parameter IN	p_user_id UUID, p_program_id UUID, p_essay_motivasi TEXT, p_data_akademik JSONB, p_data_ekonomi JSONB, p_prestasi_nonakademik JSONB
Parameter OUT	p_result VARCHAR: SUCCESS ALREADY_EXISTS PROFILE_INCOMPLETE PROGRAM_CLOSED ERROR
Logic / Pseudocode	<pre> BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE; -- 1. Cegah race condition: lock cek pengajuan aktif SELECT COUNT(*) INTO v_active_count FROM applications WHERE user_id = p_user_id AND status IN ('PENDING','TERVERIFIKASI','DITERIMA','CADANGAN') FOR UPDATE; IF v_active_count > 0 THEN ROLLBACK; p_result := 'ALREADY_EXISTS'; RETURN; END IF; -- 2. Validasi kelengkapan profil (BR-03) SELECT completion_pct INTO v_pct FROM profiles WHERE user_id = p_user_id; IF v_pct < 100 THEN ROLLBACK; p_result := 'PROFILE_INCOMPLETE'; RETURN; END IF; -- 3. Validasi program masih OPEN & kuota tersedia SELECT status, deadline, quota INTO v_prog_status, v_deadline, v_quota FROM scholarship_programs WHERE program_id = p_program_id; </pre>

```

IF v_prog_status != 'OPEN' OR CURRENT_DATE > v_deadline OR
v_quota <= 0 THEN
    ROLLBACK;
    p_result := 'PROGRAM_CLOSED';
    RETURN;
END IF;

-- 4. Insert pengajuan & generate nomor referensi (BR-10)
INSERT INTO applications (user_id, program_id, status,
essay_motivasi,
    data_akademik, data_ekonomi, prestasi_nonakademik, submitted_at,
created_at)
VALUES (p_user_id, p_program_id, 'PENDING', p_essay_motivasi,
    p_data_akademik, p_data_ekonomi, p_prestasi_nonakademik,
NOW(), NOW())
RETURNING application_id INTO v_app_id;

v_ref_no := 'BEA-' || EXTRACT(YEAR FROM CURRENT_DATE) ||
'-' ||
    LPAD(v_app_id::TEXT, 6, '0');

UPDATE applications
SET reference_no = v_ref_no
WHERE application_id = v_app_id;

-- 5. Kurangi kuota program (opsional, tergantung kebijakan bisnis)
UPDATE scholarship_programs
SET quota = quota - 1
WHERE program_id = p_program_id;

COMMIT;
p_result := 'SUCCESS';

EXCEPTION WHEN OTHERS THEN
    ROLLBACK;
    p_result := 'ERROR';
    -- Log error ke server monitoring (tidak expose ke user)
END;

```

Dipanggil oleh

POST /api/v1/applications (TSD 3.2 — FSD-2.3.3 Konfirmasi & Kirim Pengajuan)

sp_process_selection

Deskripsi singkat: Tujuannya menghitung skor kelayakan secara batch untuk seluruh kandidat TERVERIFIKASI dalam satu periode dengan transaksi atomik, memenuhi BR-08 (total bobot = 100%) dan BR-37 (≤ 30 detik untuk 1.000 kandidat)

Parameter IN

p_period VARCHAR, p_program_id UUID, p_calculated_by UUID

Parameter OUT	p_result VARCHAR: SUCCESS INVALID_WEIGHTS NO_CANDIDATE ERROR
Logic / Pseudocode	<pre> BEGIN TRANSACTION; -- 1. Ambil kuota program terlebih dahulu SELECT quota INTO v_quota FROM scholarship_programs WHERE program_id = p_program_id; -- 2. Cek ada kandidat terverifikasi IF NOT EXISTS (SELECT 1 FROM applications WHERE program_id = p_program_id AND status = 'TERVERIFIKASI') THEN ROLLBACK; p_result := 'NO_CANDIDATE'; RETURN; END IF; -- 3. Validasi bobot = 100% dari satu record sample SELECT (weight_akademik + weight_ekonomi + weight_prestasi + weight_dokumen) INTO v_total_weight FROM eligibility_scores WHERE application_id = (SELECT application_id FROM applications WHERE program_id = p_program_id AND status = 'TERVERIFIKASI' LIMIT 1); IF v_total_weight IS NULL OR v_total_weight != 100 THEN ROLLBACK; p_result := 'INVALID_WEIGHTS'; RETURN; END IF; -- 4. Loop kandidat terverifikasi FOR rec IN (SELECT application_id FROM applications WHERE program_id = p_program_id AND status = 'TERVERIFIKASI') LOOP v_skor_total := fn_calc_eligibility_score(rec.application_id); UPDATE eligibility_scores SET skor_total = v_skor_total, calculated_at = NOW(), </pre>

```

        calculated_by = p_calculated_by
        WHERE application_id = rec.application_id;
    END LOOP;

    -- 5. Generate ranking & rekomendasi
    INSERT INTO selection_results (application_id, period, rank,
    recommendation, finalized)
    SELECT
        application_id,
        p_period,
        ROW_NUMBER() OVER (ORDER BY skor_total DESC),
        CASE
            WHEN ROW_NUMBER() OVER (ORDER BY skor_total DESC)
            <= v_quota THEN 'DITERIMA'
            WHEN ROW_NUMBER() OVER (ORDER BY skor_total DESC)
            <= v_quota + 20 THEN 'CADANGAN'
            ELSE 'DITOLAK'
        END,
        FALSE
    FROM eligibility_scores e
    JOIN applications a ON e.application_id = a.application_id
    WHERE a.program_id = p_program_id AND a.status =
    'TERVERIFIKASI';

    COMMIT;
    p_result := 'SUCCESS';

    EXCEPTION WHEN OTHERS THEN
        ROLLBACK;
        p_result := 'ERROR';
    END;

```

Dipanggil oleh	POST /api/v1/selection/calculate (TSD 3.2 — FSD-2.5.1 Hitung Skor Kelayakan)
-----------------------	--

5.2 Database Functions

fn_calc_eligibility_score

Deskripsi singkat : Tujuannya menghitung skor total kelayakan berdasarkan metode SAW (Simple Additive Weighting) sesuai rumus FSD-2.5.1:

Skor Total = $(SA \times WA + SE \times WE + SP \times WP + SD \times WD) / 100$

Parameter IN	p_application_id UUID
Returns	DECIMAL(5,2)
Formula / Logic	SELECT

	<pre> ((e.skor_akademik * e.weight_akademik) + (e.skor_ekonomi * e.weight_ekonomi) + (e.skor_prestasi * e.weight_prestasi) + (e.skor_dokumen * e.weight_dokumen)) / 100.0 INTO v_total FROM eligibility_scores e WHERE e.application_id = p_application_id; RETURN v_total; </pre>
Digunakan di	sp_process_selection (TSD 5.1.b), v_selection_ranking (TSD 5.3.a)

5.3 Views

v_selection_ranking

Tujuan	Menyederhanakan query JOIN kompleks menjadi satu SELECT untuk endpoint tabel peringkat kandidat beasiswa beserta rekomendasi status pada modul Seleksi Penerima.
Tabel Sumber	applications a JOIN eligibility_scores e ON a.application_id = e.application_id JOIN users u ON a.user_id = u.user_id JOIN scholarship_programs p ON a.program_id = p.program_id LEFT JOIN selection_results s ON a.application_id = s.application_id
Kolom Output	a.application_id, a.reference_no, u.full_name, p.name AS program_name, e.skor_akademik, e.skor_ekonomi, e.skor_prestasi, e.skor_dokumen, e.skor_total, s.rank, s.recommendation, s.finalized
Digunakan oleh	GET /api/v1/selection/results (TSD 3.2 — FSD-2.5.2 Sahkan Hasil Seleksi)

v_audit_log_detail

Tujuan	Menyederhanakan query JOIN kompleks menjadi satu SELECT untuk endpoint log aktivitas sistem lengkap dengan identitas aktor pada modul Audit Log Sistem.
Tabel Sumber	audit_logs al LEFT JOIN users u ON al.actor_id = u.user_id
Kolom Output	al.log_id, u.full_name AS actor_full_name, u.email AS actor_email, al.actor_role, al.action, al.target_table, al.target_id, al.metadata, al.ip_address, al.created_at
Digunakan oleh	GET /api/v1/admin/audit-logs (TSD 3.2 — FSD-2.7.2 Audit Log Sistem)

5.4 Triggers

trg_audit_users

Event	AFTER INSERT OR UPDATE OR DELETE
Tabel	users
For Each	ROW
Logic	IF INSERT → action_type = 'USER_CREATED'; IF UPDATE → action_type = 'USER_UPDATED'; IF DELETE → action_type = 'USER_DELETED'. Insert ke audit_logs dengan kolom user_id, user_email, user_role, resource_type='users', resource_id, old_values/new_values
Tujuan	Mencatat mutasi data user ke audit log secara otomatis

trg_audit_applications

Event	AFTER UPDATE OF status
Tabel	applications
For Each	ROW
Logic	Jika OLD.status IS DISTINCT FROM NEW.status, INSERT ke audit_logs dengan action_type = 'STATUS_' NEW.status, resource_type='applications', resource_id=NEW.application_id, old_values dan new_values berisi perubahan status & admin_notes
Tujuan	Mencatat setiap perubahan status pengajuan (traceability) untuk kebutuhan investigasi Super Admin.

trg_audit_logs_immutable

Event	BEFORE UPDATE OR DELETE
Tabel	audit_logs
For Each	ROW
Logic	RAISE EXCEPTION untuk setiap percobaan UPDATE atau DELETE
Tujuan	Menjamin tabel audit_logs bersifat append-only (BR-20, AC-08) di tingkat database.

[Tambahkan sub-seksi 5.1–5.4 untuk setiap SP / function / view / trigger yang ada dalam sistem]

TSD Bagian 6 — Security Specification

6.1 Security Layers (Defense in Depth)

•

Layer Keamanan	Implementasi & Keterangan
HTTPS / TLS 1.2+	Nginx mengaktifkan HSTS (Strict-Transport-Security: max-age=31536000) dan TLS termination. Semua traffic HTTP (port 80) diarahkan otomatis ke HTTPS (301 redirect). Konfigurasi cipher suite mengikuti rekomendasi Mozilla Modern Compatibility. (Wajib — NFR-07, AC-07)
JWT Authentication	HS256 algorithm, access token berlaku 15 menit, refresh token 7 hari. Token dikirim via header Authorization: Bearer <token>. Validasi dilakukan di middleware setiap request ke protected endpoint. (Wajib — NFR-09, AC-04)
RBAC Middleware	checkRole() middleware dieksekusi setelah verifyJWT() pada setiap route. Middleware membaca klaim role dari JWT payload dan memvalidasi terhadap permission yang dibutuhkan endpoint. Pelanggaran menghasilkan HTTP 403. (Wajib — NFR-11, AC-06)
Input Validation & Sanitization	Semua input dari pengguna divalidasi menggunakan library Zod (schema validation TypeScript) di layer Controller sebelum diteruskan ke Service. Mencegah injection, type confusion, dan data malformed. (Wajib — AC-02)
Parameterized Query	Semua interaksi database dilakukan melalui Supabase Client SDK yang menggunakan parameterized query secara internal. Concatenation string langsung ke SQL dilarang keras (AC-01). Mencegah SQL Injection.
bcrypt Password Hash	Cost factor minimum 12. Proses hashing dilakukan di Service layer sebelum data diteruskan ke database. Plaintext password tidak pernah di-log atau disimpan di mana pun. (Wajib — NFR-08, AC-05)
Audit Logging	Semua event kritis (login sukses/gagal, verifikasi, seleksi, perubahan role, export) dicatat ke tabel audit_logs yang bersifat append-only via PostgreSQL trigger dan middleware. (Wajib — NFR-12, AC-08)
File Upload Security	Validasi ganda: ekstensi & MIME type di sisi klien dan server. Hanya PDF dan PNG yang diizinkan. File disimpan di Supabase Storage dengan kebijakan akses RLS. (AC-09)
Rate Limiting	Nginx membatasi jumlah request per IP untuk endpoint autentikasi (POST /auth/login, POST /auth/register) guna mencegah brute force. Dikonfigurasi dengan limit_req_zone.

AES-256 Encryption Data finansial sensitif (nomor rekening bank) dienkripsi menggunakan AES-256-CBC sebelum disimpan ke database. Kunci enkripsi disimpan sebagai environment variable, tidak pernah di-hardcode. (AC-11)

6.2 Security Specifications Detail

6.2.1 JWT Configuration

Algorithm	HS256 (HMAC-SHA256) — dikelola oleh Supabase Auth. Pemilihan HS256 karena Supabase Auth tidak mendukung konfigurasi RS256 secara native pada tier yang digunakan.
JWT Payload (Claims)	sub: UUID pengguna (user_id) email: alamat email pengguna role: 'SISWA' 'ADMIN' 'SUPER_ADMIN' (custom claim) iat: issued-at timestamp (Unix epoch) exp: expiry timestamp (iat + 900 untuk access token) jti: JWT ID unik (untuk session tracking di audit log)
Access Token Expiry	900 detik (15 menit). Token kedaluwarsa pendek untuk membatasi window of exposure jika token dicuri. Sesuai NFR-09 dan BR-41.
Refresh Token Expiry	604800 detik (7 hari). Dikelola oleh Supabase Auth. Disimpan di httpOnly cookie atau secure storage klien. Sesuai NFR-09 dan BR-41.
Secret / Signing Key	JWT Secret dikelola sepenuhnya oleh Supabase (tidak di-expose ke aplikasi). Kunci service role (SUPABASE_SERVICE_ROLE_KEY) untuk operasi admin disimpan sebagai environment variable dan TIDAK pernah di-hardcode dalam source code atau di-commit ke repository.
Token Storage di Klien	Access token disimpan di memori JavaScript (tidak di localStorage untuk mencegah XSS). Refresh token dikelola oleh Supabase Auth SDK secara internal menggunakan httpOnly cookie.
Token Refresh Flow	Ketika access token kedaluwarsa, Supabase Auth SDK secara otomatis menggunakan refresh token untuk mendapatkan access token baru tanpa memaksa pengguna login ulang. Jika refresh token juga kedaluwarsa, pengguna diarahkan ke halaman login.
Token Revocation	Saat logout, backend memanggil Supabase Auth signOut() yang menginvalidasi refresh token di sisi Supabase. Untuk kasus keamanan kritis (misal: role diubah, akun dikunci), jti dari access token yang sedang aktif dicatat di Redis sebagai blacklist hingga token tersebut kedaluwarsa secara natural.
Validasi di Middleware	1. Ambil token dari header: Authorization: Bearer <token> 2. Verifikasi signature menggunakan Supabase Auth SDK (supabase.auth.getUser(token)) 3. Periksa exp — tolak jika sudah kedaluwarsa (HTTP 401) 4. Periksa jti di Redis blacklist — tolak jika ada (HTTP 401) 5. Attach user object (id, email, role) ke request context

6. Lanjutkan ke middleware RBAC berikutnya

6.2.2 Password Security (bcrypt)

Library	Supabase Auth mengelola hashing password secara internal menggunakan bcrypt. Untuk akses API level pengguna, password tidak pernah sampai ke lapisan backend custom — langsung ditangani Supabase Auth.
Cost Factor	Minimum 12 (dikelola Supabase Auth). Cost factor 12 menghasilkan ~250ms per operasi hashing pada server modern — cukup lambat untuk membuat brute force tidak praktis, namun tidak mengganggu UX login.
Verifikasi Password	Dilakukan via Supabase Auth signInWithPassword(). Supabase membandingkan input dengan hash menggunakan bcrypt.compare() secara internal. Backend custom tidak pernah menerima atau memproses plaintext password.
Password Requirements	Minimal 8 karakter (NFR-08, ERR-REG-03) Validasi dilakukan di sisi klien (real-time strength indicator) DAN di sisi Supabase Auth Tidak ada aturan kompleksitas tambahan (uppercase, angka, simbol) pada tahap ini — dapat dikonfigurasi di Supabase Auth settings
Larangan Absolut	DILARANG: Menyimpan plaintext password di database, log, atau variabel sesi DILARANG: Mengirim password dalam response API atau log server DILARANG: Membandingkan password secara langsung tanpa fungsi time-safe comparison DILARANG: Menggunakan hashing yang lebih lemah (MD5, SHA-1, SHA-256 tanpa salt)
Password Reset Flow	Menggunakan Supabase Auth resetPasswordForEmail(). Sistem mengirimkan email dengan link reset yang mengandung token satu kali pakai. Token berlaku 1 jam. Setelah digunakan, token diinvalidasi. Seluruh proses tidak melibatkan backend custom.

6.2.3 RBAC Enforcement

Role Hierarchy	SUPER_ADMIN > ADMIN > SISWA SISWA: Pemohon beasiswa. Akses terbatas pada data milik sendiri. ADMIN: Pengelola operasional. Akses ke semua data pengajuan, verifikasi, seleksi. SUPER_ADMIN: Pemantau strategis. Akses penuh ke semua fitur termasuk audit log, konfigurasi sistem, dan evaluasi program.
Middleware Stack per Request	Request masuk → Nginx (rate limit, TLS) → verifyJWT() ← validasi token, inject user ke req.user checkRole(['ADMIN']) ← validasi role terhadap permission endpoint Controller ← proses bisnis Service ← logika bisnis Supabase SDK ← akses data (dengan RLS sebagai safety net)
verifyJWT() Middleware	Fungsi: Memvalidasi token JWT dari header Authorization. Jika tidak ada token → HTTP 401 Unauthorized Jika token tidak valid / kedaluwarsa → HTTP 401 Unauthorized Jika token ada di Redis blacklist → HTTP 401 Unauthorized Jika valid → attach req.user = { id, email, role } dan lanjut
checkRole() Middleware	Fungsi: Memvalidasi bahwa role pengguna memiliki izin untuk endpoint yang diakses. Menerima array role yang diizinkan: checkRole(['ADMIN', 'SUPER_ADMIN']) Membaca role dari req.user.role (hasil verifyJWT) Jika role tidak ada dalam daftar yang diizinkan → HTTP 403 Forbidden Jika diizinkan → lanjut ke Controller Log percobaan akses yang ditolak ke audit_logs (action_type: ACCESS_DENIED)
Contoh Penerapan Route	<pre>// Endpoint publik (tanpa auth) router.get('/programs', getActivePrograms); // Endpoint khusus Siswa router.post('/applications', verifyJWT, checkRole(['SISWA']), submitApplication); // Endpoint Admin & Super Admin router.get('/applications', verifyJWT, checkRole(['ADMIN', 'SUPER_ADMIN']), listApplications); // Endpoint eksklusif Super Admin router.get('/audit-logs', verifyJWT, checkRole(['SUPER_ADMIN']), getAuditLogs);</pre>

Role Hierarchy	SUPER_ADMIN > ADMIN > SISWA SISWA: Pemohon beasiswa. Akses terbatas pada data milik sendiri. ADMIN: Pengelola operasional. Akses ke semua data pengajuan, verifikasi, seleksi. SUPER_ADMIN: Pemantau strategis. Akses penuh ke semua fitur termasuk audit log, konfigurasi sistem, dan evaluasi program.
Middleware Stack per Request	Request masuk → Nginx (rate limit, TLS) → verifyJWT() ← validasi token, inject user ke req.user checkRole(['ADMIN']) ← validasi role terhadap permission endpoint Controller ← proses bisnis Service ← logika bisnis Supabase SDK ← akses data (dengan RLS sebagai safety net)
Row Level Security (RLS) Supabase	Sebagai safety net di lapisan database, tabel kritis (applications, profiles, fund_reports) dilindungi oleh RLS policy Supabase yang memastikan pengguna hanya dapat membaca atau memodifikasi data miliknya sendiri. RLS adalah lapisan pelengkap — bukan pengganti — validasi RBAC di middleware API.
UI Hiding (Frontend)	Menu dan komponen di UI disembunyikan berdasarkan role yang tersimpan di context aplikasi React. PENTING: UI hiding adalah penyempurnaan UX semata — BUKAN mekanisme keamanan. Semua penegakan hak akses sesungguhnya harus ada di lapisan API (middleware) dan database (RLS).

6.3 Audit Log Schema

Kolom	Tipe Data	Null?	Keterangan
log_id	UUID	NOT NULL	PRIMARY KEY, DEFAULT get_random_uuid()
user_id	UUID	NULLABLE	FK → users, SET NULL jika user dihapus — preserve log
user_email	VARCHAR(150)	NOT NULL	Denormalisasi — simpan email saat terjadi event
user_role	VARCHAR(20)	NOT NULL	'SISWA', 'ADMIN', 'SUPER_ADMIN', atau 'SISTEM'
action_type	VARCHAR(60)	NOT NULL	LOGIN_SUCCESS, LOGIN_FAILED, ROLE_CHANGED, dll

resource_type	VARCHAR(50)	NULLABLE	Nama tabel atau entitas yang menjadi objek aksi. Contoh: 'applications', 'users', 'fund_reports'.
resource_id	UUID	NULLABLE	ID entitas yang terpengaruh. Disimpan sebagai JSON untuk fleksibilitas skema.
old_values	JSONB	NULLABLE	State sebelum perubahan (UPDATE/DELETE)
new_values	JSONB	NULLABLE	State setelah perubahan (INSERT/UPDATE)
ip_address	INET	NOT NULL	Alamat IP sumber request. Menggunakan tipe INET PostgreSQL yang mendukung IPv4 dan IPv6. Wajib ada untuk analisis keamanan dan investigasi anomali.
user_agent	TEXT	NULLABLE	User-Agent string dari browser/klien pengguna. Berguna untuk investigasi forensik.
session_id	VARCHAR(255)	NULLABLE	Identifier sesi JWT (JWT ID / jti claim) untuk menghubungkan beberapa aksi dalam satu sesi login yang sama.
created_at	TIMESTAMP	NOT NULL	DEFAULT NOW()

6.3.1 Daftar Action Type

Kategori	Action Type (nilai kolom action_type)
Autentikasi	LOGIN_SUCCESS, LOGIN_FAILED, LOGOUT, EMAIL_VERIFIED, PASSWORD_CHANGED, PASSWORD_RESET_REQUESTED, ACCOUNT_LOCKED, ACCOUNT_UNLOCKED
Manajemen Pengguna	USER_CREATED, USER_UPDATED, USER_DEACTIVATED, USER_BANNED, ROLE_CHANGED, PROFILE_UPDATED
Pengajuan Beasiswa	APPLICATION_SUBMITTED, APPLICATION_CANCELLED, DOCUMENT_UPLOADED, APPLICATION_REVISION_REQUESTED
Verifikasi & Seleksi	APPLICATION_APPROVED, APPLICATION_REJECTED, SELECTION_CALCULATED, SELECTION_FINALIZED, ANNOUNCEMENT_BROADCAST

Kategori	Action Type (nilai kolom action_type)
Laporan & Pencairan	FUND_REPORT_SUBMITTED, FUND_REPORT_VERIFIED, FUND_REPORT_REVISION_REQUESTED, BANK_DATA_SUBMITTED
Konfigurasi Sistem	SYSTEM_CONFIG_UPDATED, SELECTION_WEIGHT_CONFIGURED
Data Export & Audit	DATA_EXPORTED_EXCEL, AUDIT_LOG_EXPORTED_PDF, ADMIN_PANEL_ACCESS
Proses Otomatis (Sistem)	REMINDER_EMAIL_SENT, BACKUP_COMPLETED, BACKUP_FAILED, CRON_JOB_EXECUTED

6.3.2 Mekanisme Immutability

Immutability tabel audit_logs dijamin melalui dua mekanisme yang bekerja secara paralel:

- PostgreSQL Trigger (BEFORE UPDATE OR DELETE): Trigger trg_audit_logs_immutable dipasang pada tabel audit_logs. Setiap percobaan UPDATE atau DELETE akan meng-RAISE EXCEPTION secara otomatis, memblokir operasi tersebut di tingkat database tanpa peduli siapa yang mencoba (bahkan database administrator sekalipun memerlukan tindakan manual untuk menonaktifkan trigger).
- Supabase RLS Policy: Row Level Security dikonfigurasi untuk hanya mengizinkan INSERT pada tabel audit_logs. SELECT diizinkan hanya untuk role SUPER_ADMIN. UPDATE dan DELETE diblokir total untuk semua role termasuk service_role.
- Backend Constraint: Tidak ada endpoint API yang mengekspos operasi UPDATE atau DELETE untuk tabel audit_logs. Log hanya ditulis via fungsi internal createAuditLog() yang dipanggil oleh middleware dan trigger.

6.3.3 Indexing

Untuk performa query filtering dan pagination pada halaman Audit Log Super Admin:

- CREATE INDEX idx_audit_user_id ON audit_logs(user_id);
- CREATE INDEX idx_audit_action_type ON audit_logs(action_type);
- CREATE INDEX idx_audit_created_at ON audit_logs(created_at DESC);
- CREATE INDEX idx_audit_resource ON audit_logs(resource_type, resource_id);

TSD Bagian 7 — Performance & Scalability

7.1 Response Time Targets

Tipe Operasi	Contoh Endpoint	Target Waktu	Prioritas
Auth (login/logout)	POST /api/v1/auth/login	≤ 1000 ms	Tinggi

CRUD standar	GET /api/v1/applications, PATCH /api/v1/users/{id}/profile	≤ 500 ms	Tinggi
Query dengan JOIN	GET /api/v1/selections/{programId}/results	≤ 800 ms	Menengah
Kalkulasi Skor Seleksi	POST /api/v1/selections/{programId}/run	≤ 30.000 ms (30 detik) untuk 1.000 kandidat	Tinggi
Generate Laporan PDF	GET /api/v1/reports/programs/{id}	≤ 10.000 ms (≤ 500 baris)	Rendah
Generate Laporan Excel	GET /api/v1/reports/export	≤ 5.000 ms	Rendah
Upload File	POST /api/v1/applications/{id}/documents	≤ 10.000 ms (file ≤ 5 MB, broadband ≥ 10 Mbps)	Menengah
Dashboard Admin	Dashboard Admin	≤ 7.000 ms (Lighthouse score ≥ 80)	Tinggi

7.2 Pagination Strategy

- Semua endpoint yang mengembalikan list WAJIB menggunakan pagination
- Default: 15 item per halaman | Maksimum: 100 item per halaman
- Parameter: ?page=1&per_page=15
- Response envelope harus menyertakan: {

```

    "data": [...],
    "meta": {
      "current_page": 1,
      "per_page": 15,
      "total": 247,
      "last_page": 17
    }
  }

```

Glossary

Istilah / Akronim	Definisi
FSD	Functional Specification Document — mendokumentasikan bagaimana setiap fungsi bekerja (what & how dari sisi pengguna)
TSD	Technical Specification Document — mendokumentasikan bagaimana sistem dibangun secara teknis (how dari sisi implementasi)
SRS	Software Requirements Specification — mendokumentasikan apa yang harus dilakukan sistem (what)
RBAC	Role-Based Access Control — sistem kontrol akses berdasarkan role pengguna
JWT	JSON Web Token — standar autentikasi stateless menggunakan token terenkripsi
SP	Stored Procedure — logika DB tersimpan yang dieksekusi di sisi database server
RTM	Requirements Traceability Matrix — tabel yang memetakan setiap requirement ke FSD, TSD, dan Test Case
CR	Change Request — proses formal untuk mengajukan perubahan setelah dokumen di-sign-off
ERD	Entity Relationship Diagram — diagram hubungan antar entitas dalam basis data
UUID	Universally Unique Identifier — ID unik 128-bit, lebih aman dari auto-increment integer
bcrypt	Algoritma hashing password adaptif — cost=12 artinya 2^{12} iterasi hashing
[Tambahkan]	[Istilah teknis lain yang digunakan dalam dokumen ini]